



VNiVERSiDAD
D SALAMANCA

Fundamentos e introducción al aprendizaje automático

Contenido para los estudiantes de la asignatura
de Programación Avanzada del grado en Inge-
niería Informática de la Universidad de Salamanca

Diego M. Jiménez-Bravo

Universidad de Salamanca

ÍNDICE DE CONTENIDOS

I	Sección 1 - Fundamentos	3
1	Sección 1: Fundamentos	5
1.1	¿Por qué aprender aprendizaje automático?	5
1.2	¿Qué encontrarás en esta sección?	5
2	1.1 Historia y evolución del aprendizaje automático	7
2.1	Introducción	7
2.2	Definición y el cambio de paradigma de programación	8
2.3	Raíces matemáticas y pioneros (1943-1950)	8
2.4	El nacimiento formal y la era simbólica (1956-1980)	9
2.5	Los inviernos de la IA y el resurgimiento del aprendizaje automático	9
2.6	Del <i>Deep Learning</i> a la era de los LLM (2010-presente)	10
2.7	Resumen	10
3	1.2 Conceptos fundamentales	11
3.1	Introducción	11
3.2	Datos, <i>features</i> y etiquetas	11
3.3	La tensión entre optimización y generalización: <i>overfitting</i> y <i>underfitting</i>	12
3.4	Protocolos de evaluación: validación cruzada	13
3.5	Métricas de rendimiento	13
3.6	<i>Pipeline</i> de ML: el flujo de trabajo universal	15
3.7	Resumen	16
II	Sección 2 - Aprendizaje supervisado	17
4	Sección 2: Aprendizaje supervisado	19
4.1	¿Qué es el aprendizaje supervisado?	19
4.2	Dos tipos de problemas	19
4.3	¿Qué encontrarás en esta sección?	20
5	2.1 Regresión	21
5.1	Introducción	21
5.2	Regresión lineal	21
5.3	Mínimos cuadrados ordinarios	21
5.4	Descenso del gradiente	23
5.5	Regresión polinomial y regularización	24
5.6	Validación y diagnóstico	25
5.7	Ejemplo práctico en Java con SMILE	26
5.8	Resumen	27

6	2.2 Clasificación	29
6.1	Introducción	29
6.2	Fundamentos y regresión logística	29
6.3	Algoritmos clásicos de clasificación	31
6.4	Métodos de ensamble avanzados	34
6.5	Introducción a las redes neuronales	35
6.6	Evaluación del rendimiento en clasificación	36
6.7	Ejemplo práctico en Java con SMILE	38
6.8	Resumen	39
III	Sección 3 - Aprendizaje no supervisado	41
7	Sección 3: Aprendizaje no supervisado	43
7.1	¿Qué es aprendizaje no supervisado?	43
7.2	Ejemplos y utilidades prácticas	43
7.3	¿Qué encontrarás en esta sección?	44
7.4	Nota importante	44
8	3.1 Clustering	45
8.1	Introducción	45
8.2	Algoritmos de <i>clustering</i>	45
8.3	Técnicas de evaluación en clustering	52
8.4	Ejemplo práctico en Java con SMILE	57
8.5	Resumen	58
9	3.2 Reducción de dimensionalidad	59
9.1	Introducción	59
9.2	La maldición de la dimensionalidad (<i>the curse of dimensionality</i>)	59
9.3	Métodos de proyección lineal: PCA (<i>Principal Components Analysis</i>)	60
9.4	Métodos no lineales y aprendizaje de variedades (<i>manifold learning</i>)	63
9.5	Ejemplo práctico en Java con SMILE	67
9.6	Resumen	68
10	3.3 Detección de anomalías	69
10.1	Introducción	69
10.2	Filosofía de la detección de anomalías	69
10.3	Enfoques estadísticos y de reconstrucción	70
10.4	Algoritmos específicos basados en no-supervisión	72
10.5	Resumen	74
IV	Bibliografía	75
11	Bibliografía	77
11.1	Libros de referencia del curso	77
11.2	Artículos y estudios citados	77

Bienvenido al curso **Fundamentos e introducción al aprendizaje automático** diseñado para estudiantes de la asignatura de Programación Avanzada del Grado en Ingeniería Informática de la Universidad de Salamanca.

Propósito del libro

Este libro te guía en un viaje desde los conceptos estadísticos clásicos hasta los modelos modernos de aprendizaje profundo. El objetivo es que **comprendas cómo las máquinas aprenden de los datos** y que seas capaz de construir, entrenar y evaluar modelos que resuelvan problemas reales.

Estructura del contenido

El libro se divide en tres secciones principales:

Sección 1: fundamentos

Comprender el cambio de paradigma: de la programación con reglas fijas al aprendizaje basado en datos.

- Origen histórico de la IA y el aprendizaje automático.
- Conceptos fundamentales: datos, características, etiquetas.
- Métrica de evaluación y validación cruzada.

Sección 2: aprendizaje supervisado

Aprende a modelar variables continuas (regresión) y categóricas (clasificación).

- Regresión lineal y polinomial.
- Clasificadores lineales y no lineales.
- Redes neuronales.

Sección 3: aprendizaje no supervisado

Descubre patrones ocultos en datos sin etiquetas.

- *Clustering* (agrupamiento).
- Reducción de dimensionalidad.
- Detección de anomalías.

¿A quién va dirigido?

Este curso está diseñado para:

- Estudiantes de grado en Ingeniería Informática o similares.
- Cualquier persona con interés en el aprendizaje automático y datos.
- Profesionales que desean actualizar sus habilidades.

Requisitos previos

Aunque el libro es autónomo, es útil tener:

- Conocimientos básicos de programación o disposición a aprender.
- Familiaridad con álgebra lineal y cálculo (pero no es imprescindible).
- Pensamiento analítico y paciencia con los conceptos matemáticos.

Cómo usar este libro

Cada capítulo incluye:

- **Conceptos teóricos** explicados de forma clara.
- **Ejemplos prácticos** con código Java ejecutable.
- **Visualizaciones** para entender mejor los resultados.

Se recomienda seguir el orden establecido, aunque puedes consultar secciones específicas según tus necesidades.

¡Comenzamos! Dirígete a la “Sección 1: Fundamentos” para empezar tu viaje en el aprendizaje automático.

Part I

Sección 1 - Fundamentos

SECCIÓN 1: FUNDAMENTOS

Bienvenido a la primera sección del curso. Aquí comprenderemos los principios básicos del aprendizaje automático y cómo difiere del enfoque tradicional de programación.

1.1 ¿Por qué aprender aprendizaje automático?

En la programación tradicional, un programador escribe explícitamente todas las reglas que un programa debe seguir:

```
si temperatura > 30:
    encender_aire_acondicionado()
si temperatura < 15:
    encender_calentador()
```

Pero muchos problemas del mundo real son demasiado complejos para describirlos con reglas fijas. Por ejemplo:

- ¿Cómo distinguir automáticamente un gato de un perro en una fotografía?
- ¿Cómo predecir si una transacción bancaria es fraudulenta?
- ¿Cómo clasificar correos electrónicos como *spam* o no *spam*?

El aprendizaje automático invierte el paradigma: en lugar de escribir las reglas, le enseñamos al programa a *aprender* las reglas a partir de ejemplos.

1.2 ¿Qué encontrarás en esta sección?

- **Historia y evolución:** conoceremos cómo nació la IA y cómo evolucionó hasta llegar a los modelos modernos que usamos hoy.
- **Conceptos fundamentales:** aprenderemos el vocabulario técnico esencial para trabajar con modelos de aprendizaje automático.

Estos fundamentos son críticos: sin una comprensión sólida de estos conceptos, será difícil entender las técnicas más avanzadas de las secciones posteriores.

Empecemos: dirígete al capítulo 1.1 para explorar la fascinante historia del aprendizaje automático.

1.1 HISTORIA Y EVOLUCIÓN DEL APRENDIZAJE AUTOMÁTICO

2.1 Introducción

Entender la historia del aprendizaje automático nos ayuda a comprender por qué ciertos enfoques funcionan mejor que otros y por qué la investigación actual se enfoca en ciertas direcciones.

Como se ve en la Fig. 2.1, el camino recorrido no ha sido lineal, sino una sucesión de avances matemáticos, entusiasmo, inviernos de financiación y resurgimientos.

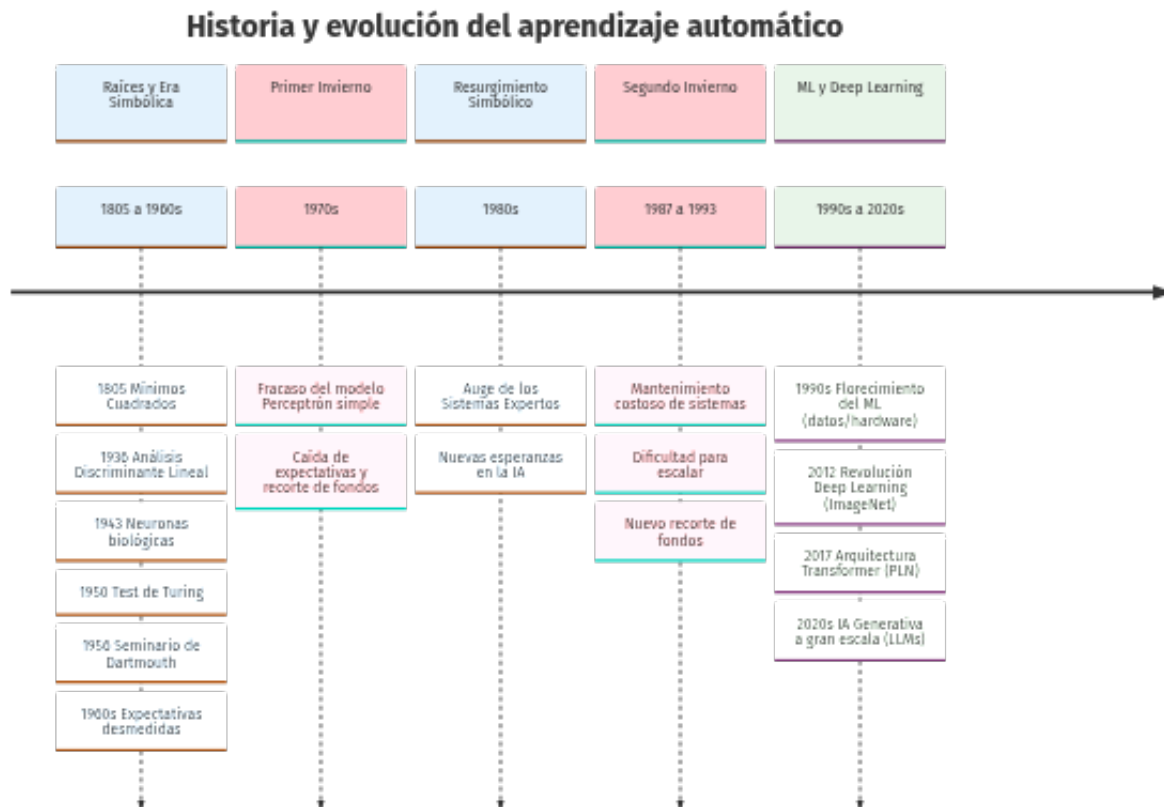


Figura2.1: Línea temporal de la historia y evolución del aprendizaje automático.

2.2 Definición y el cambio de paradigma de programación

La Inteligencia Artificial se define de manera concisa como el esfuerzo por automatizar tareas intelectuales que normalmente realizan los seres humanos. Es un campo vasto que abarca tanto el aprendizaje automático como el aprendizaje profundo, pero que también incluye métodos como la IA simbólica, que dominó el campo desde los años 50 hasta finales de los 80.

El aspecto más revolucionario del aprendizaje automático es que representa un **nuevo paradigma de programación**. En la programación clásica, un humano escribe reglas (algoritmos) que procesan datos para producir respuestas. En cambio, en el aprendizaje automático, el proceso se invierte: la máquina analiza los datos de entrada y las respuestas esperadas para inferir cuáles deberían ser las reglas estadísticamente. Para que este aprendizaje ocurra, se requieren **tres elementos clave**: puntos de datos de entrada, ejemplos de la salida esperada (etiquetas) y una forma de medir si el algoritmo está realizando un buen trabajo para ajustar el sistema (función de pérdida).

2.3 Raíces matemáticas y pioneros (1943-1950)

Aunque el término es moderno, muchos conceptos de aprendizaje estadístico se desarrollaron hace décadas. A principios del siglo XIX, Legendre y Gauss publicaron sobre el **método de mínimos cuadrados** [Gau09, Leg05], que es la forma más temprana de regresión lineal. En 1936, Fisher propuso el **análisis discriminante lineal** [Fis36] para predecir valores cualitativos.

En el ámbito computacional, el campo comenzó a gestarse en la década de 1940. En 1943, Warren McCulloch y Walter Pitts presentaron el **primer modelo computacional de neuronas biológicas** [MP43]. Posteriormente, en 1950, Alan Turing publicó su artículo fundamental “Computing Machinery and Intelligence” [Tur50], donde introdujo el **Test de Turing** como una herramienta conceptual para discutir la naturaleza de la cognición en las máquinas y la posibilidad de que estas emularan la inteligencia humana.



Figura2.2: Alan Turing en 1951, autor del artículo que introdujo el Test de Turing. Fuente: fotografía de Elliott & Fry (1951), Computer History Museum — dominio público (Wikimedia Commons, obra publicada antes de 1956).

2.4 El nacimiento formal y la era simbólica (1956-1980)

La IA como campo de investigación cristalizó formalmente en 1956, cuando John McCarthy organizó el **seminario de Dartmouth** [MMRS06]. McCarthy propuso la conjetura de que cada aspecto del aprendizaje o la inteligencia podía describirse con tal precisión que una máquina podría simularlo. Durante décadas, los expertos creyeron que la inteligencia de nivel humano se alcanzaría mediante la codificación manual de un conjunto masivo de reglas lógicas explícitas. Este enfoque, conocido como **IA simbólica**, alcanzó su auge con los **sistemas expertos** en los años 80.

2.5 Los inviernos de la IA y el resurgimiento del aprendizaje automático

La historia de la IA no ha sido lineal, sino que ha pasado por ciclos de optimismo extremo seguidos de desilusión y recortes de fondos, conocidos como “Inviernos de la IA”.

- **Primer invierno:** ocurrió en los años 70, tras el fracaso de las expectativas de la IA simbólica temprana y de los modelos simples como el **perceptrón** [Ros58], que no podían resolver problemas no lineales.
- **Segundo invierno:** sucedió a principios de los 90, cuando los sistemas expertos resultaron costosos de mantener, difíciles de escalar y limitados en su alcance.

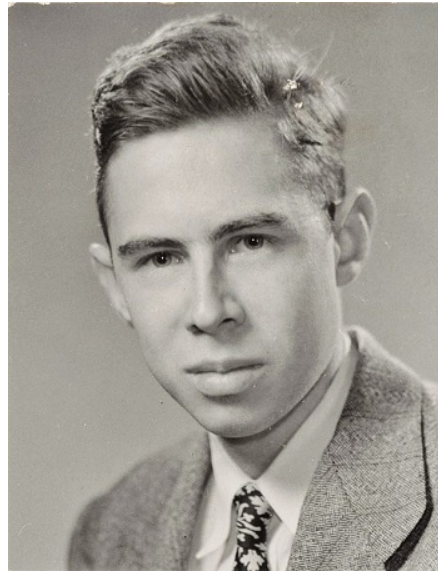


Figura2.3: Frank Rosenblatt (h. 1950), creador del Perceptrón en 1958. Fuente: Heinz Nixdorf MuseumsForum — Wikimedia Commons, licencia CC BY-SA 4.0.

El Aprendizaje Automático comenzó a florecer realmente en la década de 1990, impulsado por la disponibilidad de hardware más rápido y conjuntos de datos masivos. A diferencia de la IA simbólica, que era apta para problemas lógicos como el ajedrez, el ML (*Machine Learning*, Aprendizaje Automático en inglés) permitió abordar problemas “difusos” como el reconocimiento de voz o la clasificación de imágenes.

2.6 Del *Deep Learning* a la era de los LLM (2010-presente)

El **Aprendizaje Profundo** (*Deep Learning*) es una rama especializada del ML que enfatiza el aprendizaje de representaciones sucesivas en capas. Estas capas, estructuradas en redes neuronales, actúan como un proceso de destilación de información en múltiples etapas donde los datos se vuelven cada vez más útiles para una tarea específica. El punto de inflexión para el *Deep Learning* ocurrió entre 2011 y 2015, destacando la victoria del grupo de Hinton en el desafío **ImageNet** de 2012 [KSH12].

Durante esta época, el conjunto de datos **MNIST** (dígitos manuscritos del 0 al 9) se convirtió en el banco de pruebas por excelencia para comparar algoritmos de clasificación, desde los primeros clasificadores estadísticos hasta las redes convolucionales profundas.

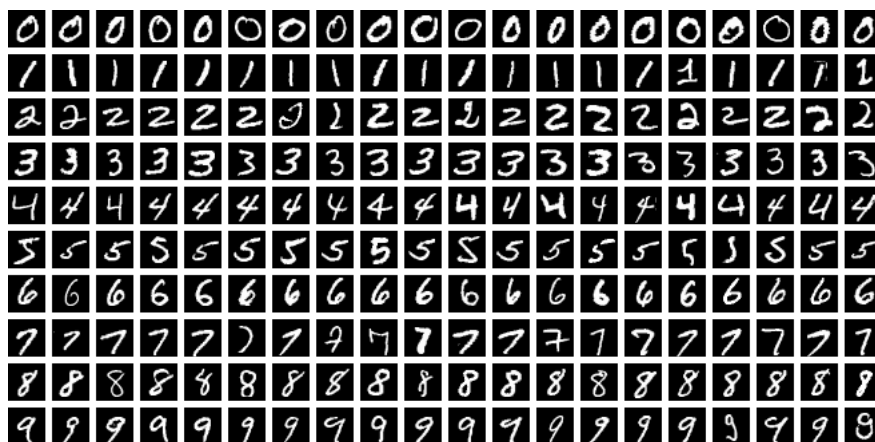


Figura2.4: Muestras del dataset MNIST: distintas variantes manuscritas de cada dígito (0-9). Fuente: Suvanjanprasai (2024) — Wikimedia Commons, licencia CC BY-SA 4.0.

A partir de 2017, la **arquitectura Transformer** [VSP+17], que utiliza un mecanismo de atención para procesar secuencias sin capas recurrentes, desató una revolución en el procesamiento de lenguaje natural. Esto permitió el desarrollo de **modelos fundacionales** entrenados mediante **aprendizaje auto-supervisado** en cantidades ingentes de datos de internet. La era actual, marcada por la **IA Generativa** (como ChatGPT y Gemini), ha llevado la tecnología a una escala de miles de millones de parámetros, permitiendo no solo clasificar, sino generar contenido creativo y funcional gracias a los LLMs (*Large Language Models*, Grandes Modelos del Lenguaje en castellano).

2.7 Resumen

- El aprendizaje automático nació como respuesta a la pregunta: ¿pueden las máquinas aprender?
- Ha pasado por ciclos de *hype* y desencanto.
- Los avances recientes se deben a más datos, mejor hardware y algoritmos innovadores.
- El aprendizaje profundo y los LLMs son el estado del arte actual.

Siguiente: en el capítulo 1.2, aprenderemos los conceptos fundamentales que necesitas para trabajar con cualquier modelo de aprendizaje automático.

1.2 CONCEPTOS FUNDAMENTALES

3.1 Introducción

Para trabajar efectivamente con modelos de aprendizaje automático, necesitamos dominar cierta terminología y comprensión de conceptos clave. Este capítulo construye el vocabulario que usaremos a lo largo del curso.

3.2 Datos, *features* y etiquetas

El éxito de cualquier sistema de ML depende de cómo se represente la información.

Muestra	Feature	Feature	Feature	Label
	1	2	3	
Ejemplo	Edad	Peso	Altura	Tipo
1	25	70	175	Gato
2	8	4	90	Gato
3	35	80	180	Perro

- **Muestra** (*sample*): es la unidad básica de información, también denominada punto de datos, instancia o ejemplo. En un conjunto de datos tabular, cada fila representa una muestra individual.
- **Atributos y *features***: las muestras se caracterizan por sus *features* (características o atributos), que son variables cuantitativas o cualitativas que miden diferentes aspectos de la instancia. Técnicamente, un “atributo” es un tipo de dato (ej. “edad”), mientras que una “*feature*” suele referirse al atributo más su valor específico (ej. “edad = 25”). En *deep learning*, todas las entradas se vectorizan para ser procesadas como puntos en un espacio geométrico.
- **Etiqueta** (*label*) o *target*: en el aprendizaje supervisado, cada muestra tiene asociada una respuesta correcta denominada etiqueta u objetivo. El conjunto de etiquetas para todo un *dataset* constituye la “**verdad fundamental**” (*ground-truth*). En problemas de clasificación, el objetivo es predecir una categoría (clase), mientras que en regresión se busca un valor numérico continuo o escalar.

```
import java.util.List;

// Características: pueden ser números, categorías, etc.
List<String> frutas = List.of("metros_cuadrados", "numero_habitaciones", "ubicacion",
    ↪ "año_construccion");
```

(continues on next page)

(continued from previous page)

```
// Etiqueta: lo que queremos predecir  
String label = "precio";
```

3.3 La tensión entre optimización y generalización: *overfitting* y *underfitting*

El problema fundamental del ML es la tensión entre ajustar el modelo a los datos conocidos y su capacidad de actuar sobre datos nuevos.

- **Optimización:** es el proceso de ajustar los parámetros de un modelo (pesos) para obtener el mejor rendimiento posible en los datos de entrenamiento.
- **Generalización:** se refiere a qué tan bien funciona el modelo entrenado con datos que nunca ha visto antes
- **Overfitting (sobreajuste):** ocurre cuando un modelo es demasiado complejo en relación con la cantidad y el ruido de los datos. El modelo “memoriza” el ruido o patrones aleatorios del entrenamiento que no existen en los datos reales, lo que provoca una pérdida baja en entrenamiento pero muy alta en validación.
- **Underfitting (subajuste):** sucede cuando el modelo es demasiado simple para capturar la estructura subyacente de los datos. En este estado, tanto el error de entrenamiento como el de validación son elevados.
- **Trade-off sesgo-varianza:** el error de generalización se divide en:
 - **Sesgo (bias):** error por suposiciones erróneas (ej. asumir una relación lineal cuando es cuadrática); un sesgo alto causa *underfitting*.
 - **Varianza:** error por excesiva sensibilidad a pequeñas variaciones en el entrenamiento; una varianza alta causa *overfitting*.

La Fig. 3.1 ilustra los tres escenarios ajustando distintos modelos a los mismos datos: una recta demasiado simple (izquierda), un polinomio de grado moderado que sigue la tendencia real (centro) y un polinomio de grado muy alto que memoriza cada punto, incluido el ruido (derecha).

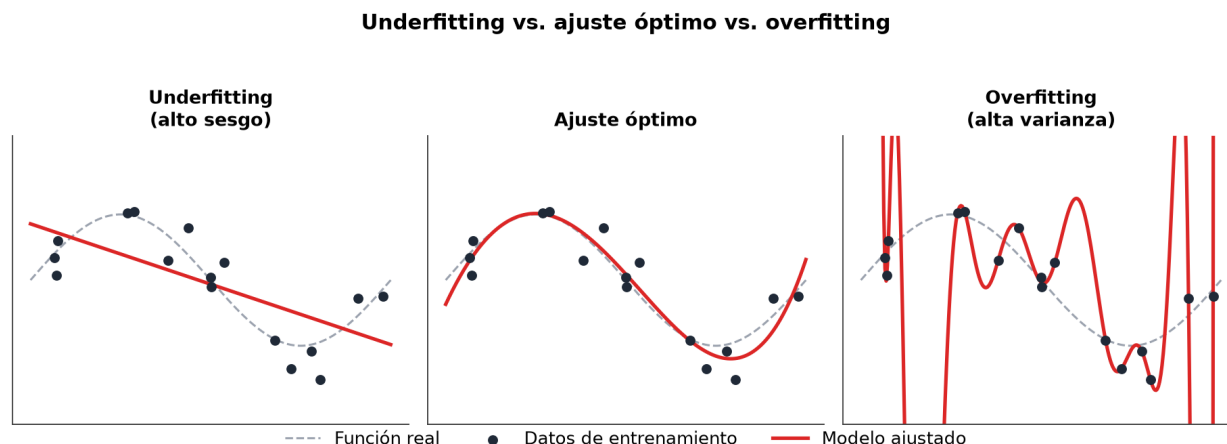


Figura3.1: *Underfitting* (alto sesgo) vs. ajuste óptimo vs. *overfitting* (alta varianza).

3.4 Protocolos de evaluación: validación cruzada

Para medir la generalización de forma fiable, no basta con evaluar el modelo sobre los mismos datos de entrenamiento.

- **División de datos:** la práctica estándar es dividir los datos en tres conjuntos: entrenamiento (para aprender los pesos), validación (para elegir hiperparámetros y evitar el “leakage” de información) y prueba (para la evaluación final e imparcial).
- **Validación cruzada K -fold:** cuando los datos son escasos, la división simple puede ser poco representativa. Este método consiste en dividir los datos en K particiones (normalmente 5 o 10). El modelo se entrena K veces; en cada iteración, se usa una partición distinta para validación y las $K - 1$ restantes para entrenamiento. El puntaje final es el promedio de los K resultados obtenidos, lo que reduce la varianza de la evaluación.
- **Estratificación:** en clasificación, es vital que cada “fold” mantenga la misma proporción de clases que el dataset original para evitar sesgos, proceso denominado K -fold estratificado.

La Fig. 3.2 muestra un ejemplo con $K = 5$: en cada fila (cada “fold”), un bloque distinto de datos actúa como conjunto de validación (en rojo) mientras el resto se usa para entrenar (en azul).

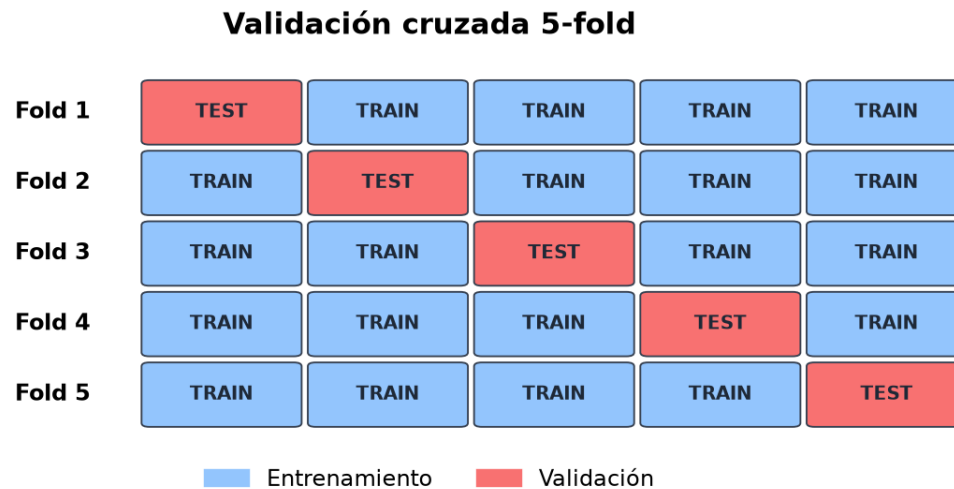


Figura3.2: Validación cruzada K -fold con $K = 5$: cada bloque de datos actúa una vez como conjunto de validación.

3.5 Métricas de rendimiento

Las métricas permiten cuantificar el éxito del modelo y guiar las decisiones técnicas.

- **Matriz de confusión:** es una herramienta bidimensional utilizada para evaluar de forma detallada el rendimiento de un clasificador. Su estructura consiste en una tabla donde cada fila representa la clase real (*ground-truth*) y cada columna representa la clase predicha por el modelo. En un problema de clasificación binaria (dos clases), la matriz contiene cuatro valores fundamentales que permiten desglosar los aciertos y errores del sistema:
 - **Verdaderos Positivos** (TP, *True Positives*): son las instancias que el modelo clasificó correctamente como positivas.
 - **Verdaderos Negativos** (TN, *True Negatives*): son las instancias que el modelo clasificó correctamente como negativas.

- **Falsos Positivos** (FP, *False Positives*): ocurre cuando el modelo predice incorrectamente que una instancia es positiva, cuando en realidad es negativa. Este tipo de error se conoce también como error de tipo I o “falsa alarma”.
- **Falsos Negativos** (FN, *False Negatives*): ocurre cuando el modelo predice incorrectamente que una instancia es negativa, cuando en realidad es positiva. Se denomina también error de tipo II o “omisión”.

La Fig. 3.3 resume visualmente estos cuatro valores: la diagonal principal (verde) representa los aciertos del modelo, mientras que la diagonal secundaria (roja) representa sus dos tipos de error.

		Predicción	
		Positivo	Negativo
Clase real	Positivo	TP Verdadero Positivo	FN Falso Negativo
	Negativo	FP Falso Positivo	TN Verdadero Negativo

Figura3.3: Matriz de confusión: aciertos (verde) y errores de tipo I y II (rojo).

- **Precisión** (*accuracy*): fracción de predicciones correctas sobre el total. Puede ser engañosa si las clases están desbalanceadas. $Accuracy = \frac{TP+TN}{TP+TN+FP+FN}$
- **Precisión** (**precisión de clase**): capacidad del modelo para no etiquetar como positiva una muestra que es negativa. $Precisión = \frac{TP}{TP+FP}$
- **Recall** (**exhaustividad/sensibilidad**): capacidad del modelo para encontrar todas las muestras positivas ($TP/(TP+FN)$). $Recall = \frac{TP}{TP+FN}$
- **F1-score**: media armónica entre Precisión y Recall; es útil cuando se busca un equilibrio entre ambas, penalizando valores extremos. $F1 = 2 \times \frac{Precisión \times Recall}{Precisión + Recall}$
- **AUC-ROC**: el área bajo la curva de la característica operativa del receptor representa la probabilidad de que el modelo clasifique una muestra positiva aleatoria por encima de una negativa. Un AUC de 1.0 es perfecto, mientras

que 0.5 equivale a una clasificación aleatoria.

3.6 Pipeline de ML: el flujo de trabajo universal

El desarrollo de un proyecto de Aprendizaje Automático sigue un *blueprint* universal que garantiza que el sistema no solo aprenda de los datos, sino que sea capaz de generalizar sus resultados en entornos de producción. La Fig. 3.4 resume las 10 etapas y su naturaleza cíclica: el ajuste de hiperparámetros suele requerir volver a entrenar el modelo varias veces antes de la evaluación final.

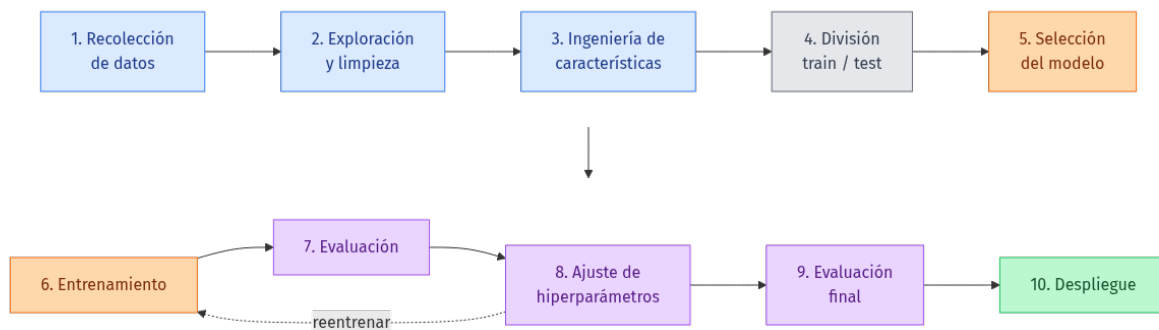


Figura3.4: El *pipeline* de Machine Learning: 10 etapas desde la recolección de datos hasta el despliegue.

A continuación, se definen las etapas del *pipeline* de ML integrando los fundamentos teóricos y técnicos de las fuentes:

- 1. Recolección de datos:** esta es la fase más ardua y costosa, donde se debe entender el dominio del problema y los objetivos de negocio. Consiste en obtener muestras representativas del entorno real, lo que a menudo implica la anotación o etiquetado manual por parte de expertos humanos para generar la “verdad fundamental” (*ground-truth*) necesaria en el aprendizaje supervisado.
- 2. Exploración y limpieza:** es una mala práctica tratar al dataset como una caja negra; se debe visualizar la distribución de los datos mediante histogramas o mapas para detectar anomalías y valores atípicos (*outliers*). La limpieza incluye el tratamiento de valores faltantes (ya sea eliminando registros o imputando promedios) y la detección de errores de etiquetado para evitar que el ruido degrade el rendimiento del modelo.
- 3. Ingeniería de características:** consiste en aplicar el conocimiento humano para realizar transformaciones no aprendidas que faciliten la tarea del algoritmo, como normalizar escalas numéricas o estandarizar textos para borrar diferencias de codificación. Aunque el *Deep Learning* automatiza gran parte de este proceso, una buena ingeniería de características permite resolver problemas con muchos menos datos y recursos computacionales.
- 4. División *train/test*:** para evaluar la capacidad de generalización, los datos deben dividirse estrictamente en conjuntos de entrenamiento, validación y prueba. Es crítico realizar un barajado aleatorio (*shuffling*) para asegurar representatividad, excepto en series temporales, donde el conjunto de prueba debe ser cronológicamente posterior al de entrenamiento para evitar fugas de información del futuro.
- 5. Selección del modelo:** en esta etapa se eligen los prejuicios de arquitectura (*architecture priors*) adecuados para la tarea, como redes convolucionales para imágenes o Transformers para lenguaje natural. El primer objetivo técnico es desarrollar un modelo sencillo que logre superar un baseline de sentido común, demostrando que existe un patrón estadístico explotable en los datos.
- 6. Entrenamiento:** el entrenamiento es un proceso iterativo llamado bucle de entrenamiento, donde los pesos del modelo se inicializan aleatoriamente y se ajustan gradualmente. Mediante el paso hacia adelante (*forward pass*), el

cálculo de la pérdida y la retropropagación del gradiente, el sistema optimiza sus parámetros internos para minimizar el error en los datos de entrenamiento.

7. **Evaluación:** se utiliza el conjunto de validación para monitorear el rendimiento del modelo durante el entrenamiento mediante métricas como la precisión, *recall* o el AUC. Esta fase permite detectar si el modelo está en un estado de subajuste (*underfitting*) o si ha comenzado a memorizar el ruido de los datos de entrenamiento.
8. **Ajuste de hiperparámetros:** los hiperparámetros son configuraciones externas (como el número de capas o la tasa de aprendizaje) que no se aprenden mediante gradiente descendente, sino mediante búsqueda sistemática. Se emplean técnicas como la validación cruzada *K-fold* o la optimización bayesiana para encontrar la configuración que maximice la generalización antes de la evaluación final.
9. **Evaluación final:** una vez seleccionado el mejor modelo y ajustados sus hiperparámetros, se realiza una única evaluación sobre el conjunto de prueba, el cual debe haber permanecido en una “bóveda” sin ser consultado previamente. Si los resultados aquí son significativamente peores que en validación, es señal de que hubo un sobreajuste al proceso de validación o una falta de representatividad en los datos.
10. **Despliegue:** el modelo final se exporta para su uso en producción, ya sea como una API web, en aplicaciones móviles o dispositivos embebidos. El *pipeline* no termina aquí; es imperativo implementar un monitoreo constante, dado que los datos reales suelen degradarse con el tiempo (*concept drift*), requiriendo ciclos de reentrenamiento periódico.

3.7 Resumen

- **Muestra:** un punto individual en el conjunto de datos.
- **Características:** variables de entrada que describen una muestra.
- **Etiqueta:** variable de salida que queremos predecir.
- **Overfitting:** memorizar datos de entrenamiento, mal en prueba.
- **Underfitting:** modelo demasiado simple, mal en ambos.
- **Validación cruzada:** técnica para evaluar desempeño sin sesgo.
- **Precisión:** ratio de positivos predichos correctamente.
- **Recall:** ratio de positivos reales encontrados.
- **Pipeline de ML:** se compone de 10 etapas, desde la recolección de datos al despliegue en producción.

Siguiente: ahora que entiendes los fundamentos, pasaremos a la Sección 2: Aprendizaje Supervisado, donde aprenderás a construir modelos prácticos.

Part II

Sección 2 - Aprendizaje supervisado

SECCIÓN 2: APRENDIZAJE SUPERVISADO

Bienvenido a la segunda sección del curso. Aquí aprenderemos a construir modelos que aprenden a partir de ejemplos etiquetados.

4.1 ¿Qué es el aprendizaje supervisado?

En aprendizaje supervisado, tenemos:

- **Datos:** características de entrada (X).
- **Etiquetas:** respuestas correctas (y).
- **Objetivo:** aprender la relación $X \rightarrow y$.

Es como aprender con un “supervisor” que te dice si tus respuestas son correctas.

4.2 Dos tipos de problemas

4.2.1 1. Regresión

Predecimos un **valor continuo** (un número real):

- Predecir el precio de una casa.
- Predecir la temperatura de mañana.
- Predecir la cantidad de lluvia.
- Predecir el crecimiento de una población.

4.2.2 2. Clasificación

Predecimos una **categoría** o **clase**:

- Diagnosticar si una persona tiene una enfermedad (sí/no).
- Clasificar emails como *spam* o no *spam*.
- Reconocer si una imagen es un gato, perro o pájaro.
- Predecir si un cliente va a abandonar.

4.3 ¿Qué encontrarás en esta sección?

- **Regresión:**
 - Regresión lineal: el modelo más simple y fundamental.
 - Mínimos cuadrados ordinarios: cómo entrena una regresión lineal.
 - Descenso del gradiente: el algoritmo de optimización más importante.
 - Regresión polinomial y regularización: cómo hacer modelos más complejos.
- **Clasificación:**
 - Regresión logística: clasificación binaria.
 - Matriz de confusión y métricas: cómo evaluar clasificadores.
 - Árboles de decisión y *Random Forests*: modelos interpretables.
 - *Support Vector Machines*: separación óptima de clases.
 - Métodos *ensemble*: combinar múltiples modelos.
 - Redes neuronales: introducción a las redes profundas.

Empecemos: dirígete al Capítulo 2.1 para aprender sobre regresión, la base de muchos modelos de aprendizaje automático.

2.1 REGRESIÓN

5.1 Introducción

La regresión es el problema de predecir un **valor continuo**. En otras palabras, la regresión predice una respuesta cuantitativa o valor escalar continuo Y a partir de una o más **variables predictoras** X .

5.2 Regresión lineal

La regresión lineal es una técnica fundamental que asume una relación aproximadamente lineal entre los predictores y la variable objetivo.

- **Simple vs. múltiple:** en la **regresión lineal simple** se utiliza un único predictor para modelar $Y \approx \beta_0 + \beta_1 X$. La **regresión lineal múltiple** extiende este concepto asignando un coeficiente o peso específico a cada una de las p características disponibles: $Y = \beta_0 + \beta_1 X_1 + \beta_2 X_2 + \dots + \beta_p X_p + \epsilon$.
- **Parámetros del modelo:** los coeficientes β_0 (intercepto o sesgo) y $\beta_1 \dots \beta_p$ (pesos o pendientes) representan el conocimiento aprendido por el modelo a partir de los datos. Técnicamente, en un espacio de entrada de p dimensiones, este modelo representa un **hiperplano**.

La Fig. 5.1 muestra un ejemplo con un único predictor: la recta roja es el ajuste OLS (*Ordinary Least Squares*) y las líneas grises verticales representan los residuos (la distancia entre cada punto y la predicción).

Un ejemplo clásico de la literatura [JWHT13] es el conjunto de datos *Advertising*, donde se ajusta una recta que predice las ventas (*Sales*) a partir de la inversión en publicidad en TV (*TV*):

5.3 Mínimos cuadrados ordinarios

El método de mínimos cuadrados ordinarios (OLS, *Ordinary Least Squares*) es el enfoque estándar para entrenar modelos lineales, buscando los parámetros que mejor se ajustan a los datos de entrenamiento.

- **Residuales y RSS:** un residual $e_i = y_i - \hat{y}_i$ es la diferencia entre el valor real y el predicho. El objetivo de OLS es minimizar la **suma de los cuadrados de los residuales** (RSS, *Residual Sum of Squares*): $RSS = \sum_{i=1}^n (y_i - \hat{y}_i)^2$.
- **Solución analítica** (ecuaciones normales): a diferencia de otros modelos complejos, el modelo lineal tiene una solución única y global que puede calcularse directamente mediante álgebra matricial: $\hat{\beta} = (X^T X)^{-1} X^T y$.
- **Interpretación geométrica:** el vector de predicciones $\hat{y}$ es la **proyección ortogonal** del vector de observaciones y sobre el subespacio lineal definido por las columnas de la matriz de características X .

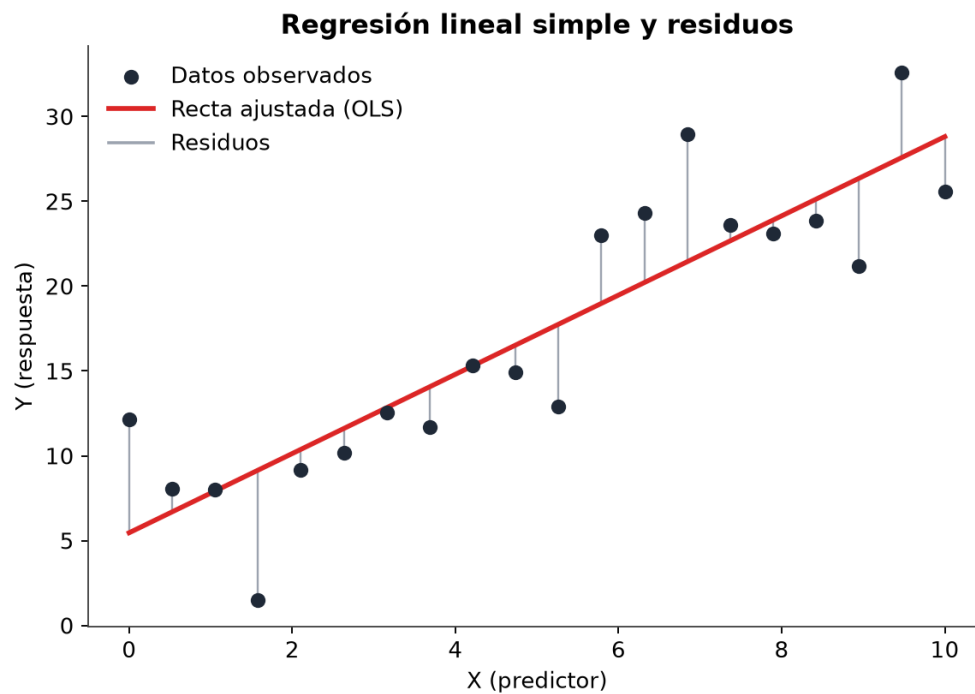


Figura5.1: Regresión lineal simple: recta ajustada y residuos.

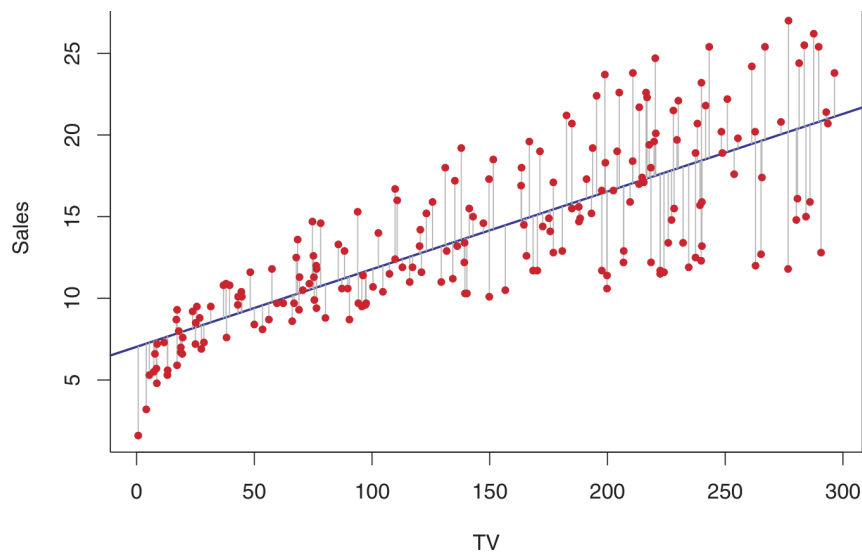


Figura5.2: Ajuste por mínimos cuadrados de *Sales* sobre *TV* en el conjunto de datos *Advertising*. Fuente: James, G., Witten, D., Hastie, T., & Tibshirani, R. (2013). *An Introduction to Statistical Learning*, Figura 3.1. Springer. Libro de libre distribución para uso educativo (statlearning.com).

5.4 Descenso del gradiente

Para conjuntos de datos masivos que no caben en memoria o modelos con miles de parámetros, resolver la ecuación normal es computacionalmente costoso. El descenso del gradiente (GD, *Gradient Descent*) es el algoritmo de optimización iterativo que impulsa el aprendizaje automático moderno.

- **Mecánica de optimización:** el algoritmo inicializa los pesos aleatoriamente y los ajusta gradualmente midiendo el gradiente local de la función de pérdida con respecto a los parámetros. Se mueve en la dirección opuesta al gradiente (hacia “abajo” en la curva de error) hasta alcanzar un mínimo.
- **Tasa de aprendizaje (η):** es un hiperparámetro crítico que determina el tamaño del paso en cada iteración. Una tasa demasiado alta puede causar divergencia, mientras que una muy baja hará que el entrenamiento sea excesivamente lento.
- **Variantes:**
 - *Batch* GD: utiliza todo el conjunto de entrenamiento para cada paso.
 - *Stochastic* GD (SGD): utiliza una sola muestra aleatoria por paso, lo que lo hace mucho más rápido y capaz de manejar flujos de datos en línea.
 - *Mini-batch* GD: procesa pequeños grupos de datos (*batches*), equilibrando la estabilidad del *Batch* GD y la eficiencia del SGD.
- **Momentum:** técnica inspirada en la física que ayuda a evitar mínimos locales y acelera la convergencia al tener en cuenta las actualizaciones de peso anteriores.

Como se ve en la Fig. 5.3, el algoritmo parte de un punto inicial y va dando pasos sucesivos en la dirección de descenso más pronunciada hasta converger al mínimo de la función de pérdida.

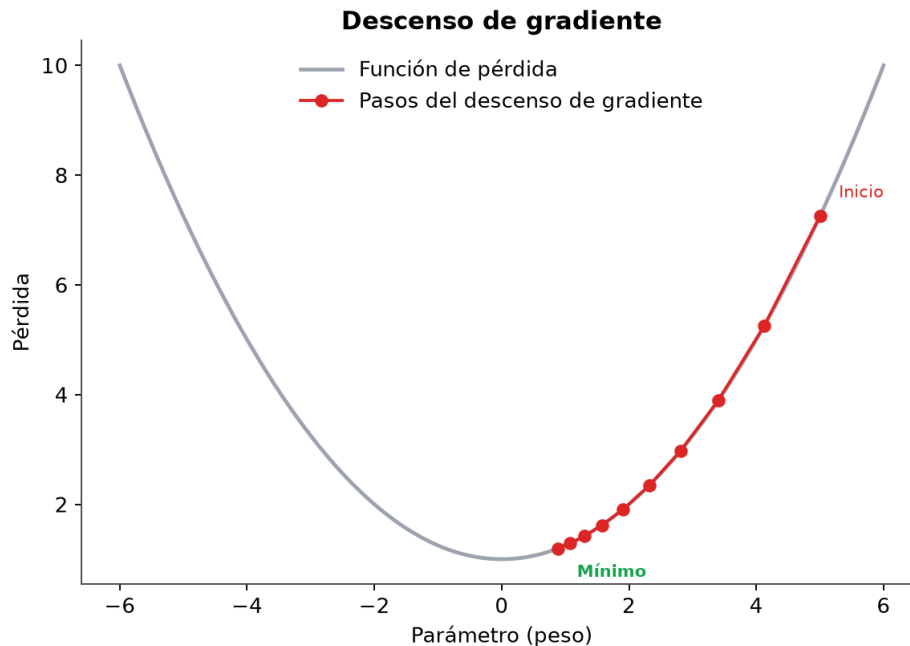


Figura 5.3: Descenso de gradiente: pasos iterativos hacia el mínimo de la pérdida.

5.5 Regresión polinomial y regularización

Cuando los datos presentan curvas, se pueden extender los modelos lineales mediante ingeniería de características.

- **Regresión polinomial:** se añaden potencias de los predictores originales (ej. X^2 , X^3) como nuevas características. Aunque el modelo es no lineal respecto a las entradas originales, sigue siendo un modelo lineal en sus parámetros, lo que permite usar OLS para entrenarlo.
- **El riesgo del sobreajuste:** aumentar el grado del polinomio aumenta la flexibilidad del modelo, lo que puede llevarlo a memorizar el ruido de los datos de entrenamiento (*overfitting*) y fallar en la generalización.
- **Técnicas de regularización:** consisten en añadir una penalización a la función de costo por tener pesos grandes, forzando al modelo a ser más simple.
 - **Ridge (L2)** [HK70]: añade una penalización proporcional al cuadrado de los pesos. Encoge los coeficientes hacia cero pero nunca los elimina por completo; funciona muy bien cuando hay muchos predictores correlacionados.
 - **Lasso (L1)** [Tib96]: añade una penalización proporcional al valor absoluto de los pesos. Tiene la propiedad única de forzar algunos coeficientes a ser exactamente cero, realizando automáticamente selección de variables.
 - **Elastic Net:** una combinación de *Ridge* y *Lasso* que utiliza ambos tipos de penalización mediante un ratio de mezcla.

La Fig. 5.4 compara cómo evolucionan los coeficientes de un modelo al aumentar la penalización λ : Ridge los encoge suavemente sin llegar nunca a cero, mientras que Lasso los lleva a cero de forma exacta, seleccionando variables.

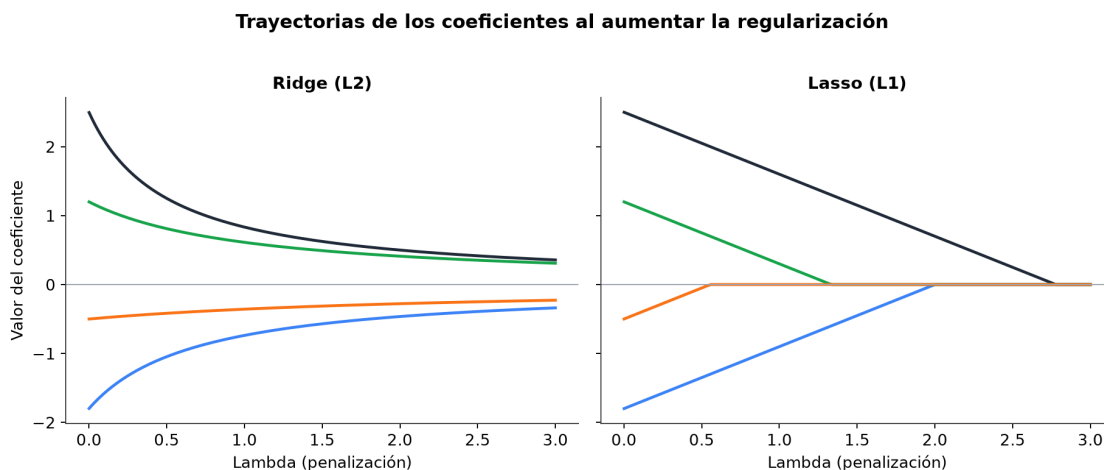


Figura 5.4: Trayectorias de los coeficientes: Ridge (encogimiento suave) vs. Lasso (selección de variables).

Sobre datos reales, la Fig. 5.5 muestra cómo evolucionan los coeficientes estandarizados de Ridge para el conjunto de datos *Credit*, tanto en función de λ (izquierda) como de la norma relativa del coeficiente (derecha).

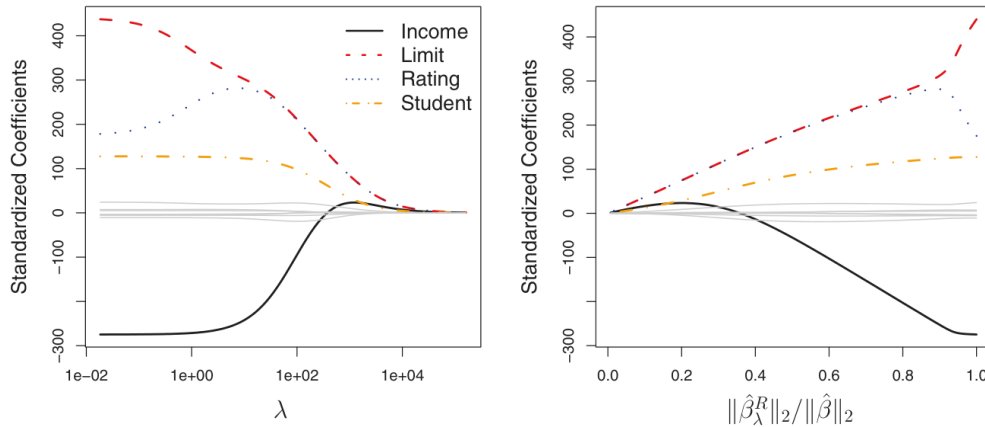


Figura 5.5: Coeficientes de regresión Ridge estandarizados para el conjunto de datos *Credit*. Fuente: James, G., Witten, D., Hastie, T., & Tibshirani, R. (2013). *An Introduction to Statistical Learning*, Figura 6.4. Springer. Libro de libre distribución para uso educativo (statlearning.com).

5.6 Validación y diagnóstico

Evaluar un modelo de regresión requiere métricas específicas y el análisis de la calidad del ajuste.

- Métricas de error:
 - **MSE (Mean Squared Error)**: promedio de los errores al cuadrado; penaliza fuertemente las desviaciones grandes. $MSE = \frac{1}{n} \sum_{i=1}^n (y_i - \hat{y}_i)^2$
 - **RMSE (Root MSE)**: raíz cuadrada del MSE, lo que devuelve el error a las mismas unidades que la variable objetivo. $RMSE = \sqrt{\frac{1}{n} \sum_{i=1}^n (y_i - \hat{y}_i)^2}$
 - **MAE (Mean Absolute Error)**: promedio de los valores absolutos de los errores; es más robusto ante valores atípicos (*outliers*). $MAE = \frac{1}{n} \sum_{i=1}^n |y_i - \hat{y}_i|$
- **Coefficiente de determinación (R2)**: mide la proporción de la varianza en Y que es explicada por el modelo. Un valor de 1 indica un ajuste perfecto, mientras que 0 indica que el modelo no es mejor que predecir siempre la media.
- **Análisis de residuos**: graficar los residuos versus los valores ajustados permite detectar problemas como la no-linealidad. Si el gráfico muestra un patrón (como una curva en forma de U), indica que el modelo lineal es insuficiente.
- **Curvas de validación**: las curvas de validación son herramientas gráficas fundamentales para diagnosticar el comportamiento del modelo frente a su complejidad.
 - **Diagnóstico de sesgo y varianza**: al graficar el error de entrenamiento y de validación contra la complejidad del modelo (ej. grado del polinomio), podemos identificar:
 - * **Bajo ajuste (high bias)**: errores altos tanto en entrenamiento como en validación.
 - * **Sobreajuste (high varianza)**: error de entrenamiento muy bajo pero error de validación muy alto; existe una brecha significativa entre ambas curvas.
 - **Regla de la desviación estándar**: en la práctica, se suele elegir el modelo más simple que esté dentro de una desviación estándar del error mínimo en la curva de validación para asegurar la parsimonia.

5.7 Ejemplo práctico en Java con SMILE

Los conceptos de esta sección se pueden poner en práctica con **SMILE** (*Statistical Machine Intelligence and Learning Engine*), una librería de aprendizaje automático para Java. El repositorio `programacion-avanzada-smile` de la asignatura de Programación Avanzada incluye el ejemplo completo `Ejemplo04Regresion.java`, del que se extrae el siguiente fragmento.

El conjunto `viviendas.csv` es sintético: el precio se ha generado con una relación lineal conocida más ruido gaussiano ($\text{precio} = 1.8 \cdot \text{superficie} + 12 \cdot \text{habitaciones} + 9 \cdot \text{banos} - 1.1 \cdot \text{antiguedad} - 6.5 \cdot \text{distancia_centro} + 55 + \varepsilon$), de modo que se puede comprobar si OLS recupera los coeficientes reales.

```
import smile.data.DataFrame;
import smile.data.formula.Formula;
import smile.io.Read;
import smile.regression.LASSO;
import smile.regression.LinearModel;
import smile.regression.OLS;
import smile.regression.RidgeRegression;
import smile.validation.CrossValidation;
import smile.validation.metric.R2;
import smile.validation.metric.RMSE;

DataFrame viviendas = Read.csv("data/viviendas.csv", "header=true");
Formula formula = Formula.lhs("precio"); // precio ~ resto de columnas

// Utiles.split permuta las filas y las reparte en entrenamiento/test
Utiles.Particion particion = Utiles.split(viviendas, 0.75);
DataFrame train = particion.train();
DataFrame test = particion.test();

// 1. Minimos cuadrados ordinarios: solucion exacta en un solo paso
LinearModel ols = OLS.fit(formula, train);
System.out.println(ols); // coeficientes, p-valores, R2... (como summary() en R)
System.out.printf("Termino independiente: %.3f (real: 55)%n", ols.intercept());

// 2. Evaluacion sobre el conjunto de test
double[] yTest = formula.y(test).toDoubleArray();
double[] prediccion = ols.predict(test);
System.out.printf("RMSE = %.3f%n", RMSE.of(yTest, prediccion));
System.out.printf("R2 = %.4f%n", R2.of(yTest, prediccion));

// 3. Regularizacion: Ridge encoge los coeficientes, LASSO puede anularlos
LinearModel ridge = RidgeRegression.fit(formula, train, 0.1);
LinearModel lasso = LASSO.fit(formula, train, new LASSO.Options(0.5));
System.out.printf("Ridge RMSE = %.3f%n", RMSE.of(yTest, ridge.predict(test)));
System.out.printf("LASSO RMSE = %.3f%n", RMSE.of(yTest, lasso.predict(test)));

// 4. Validacion cruzada de 10 pliegues
var cv = CrossValidation.regression(10, formula, viviendas,
    (f, datos) -> OLS.fit(f, datos));
System.out.printf("OLS RMSE medio = %.3f | R2 medio = %.4f%n",
    cv.avg().rmse(), cv.avg().r2());
```

Algunas ideas que conviene observar al ejecutarlo:

- `Formula.lhs("precio")` declara la variable objetivo; el resto de columnas del `DataFrame` se usan como predictoras.

- `OLS.fit` resuelve la ecuación normal de mínimos cuadrados de forma exacta, y el modelo resultante imprime los coeficientes estimados junto a su error estándar y su p -valor.
- El ejemplo completo compara también OLS con *Random Forest* y *Gradient Boosting*: con datos generados por un modelo lineal, OLS gana a los modelos basados en árboles, lo que ilustra que «más complejo» no significa «mejor».

Para ejecutarlo desde la raíz del repositorio: `mvn exec:java -Dexec.mainClass=es.usal.smile.Ejemplo04Regresion.`

5.8 Resumen

- **Regresión Lineal:** modelo simple pero poderoso.
- **OLS:** solución exacta, pero costosa computacionalmente
- **Descenso de gradiente:** método general, funciona para cualquier modelo.
- **Regresión polinomial:** captura relaciones no lineales.
- **Regularización:** previene *overfitting* (*Ridge* para retención, *Lasso* para selección).
- **Validación:** métricas específicas para validar la regresión acompañadas de un estudio del proceo de entrenamiento.

Siguiente: en el siguiente capítulo, aprenderemos a extender estas ideas a **Clasificación**, donde predecimos categorías en lugar de números continuos.

2.2 CLASIFICACIÓN

6.1 Introducción

En clasificación, nuestro objetivo es predecir **categorías** (clases) en lugar de valores continuos. Para clasificar una muestra o instancia con una clase específica los algoritmos de clasificación se basan en los atributos de la muestra. Algunos ejemplos típicos incluyen:

- Clasificación de correos electrónicos: *spam* vs. *no-spam*.
- Clasificación médica: enfermo vs. sano.
- Clasificación de especies: gato vs. perro, etc.
- Etc.

6.2 Fundamentos y regresión logística

A diferencia de la regresión lineal, que predice valores continuos, la clasificación comienza estimando la probabilidad de pertenencia a una categoría.

- **Mecánica del modelo:** la regresión logística utiliza la **función sigmoide** (o logística) para transformar una combinación lineal de las características de entrada en un valor entre 0 y 1. La fórmula se define como: $\sigma(t) = \frac{1}{1+e^{-t}}$

La entrada t de esta función se denomina **logit** (o *log-odds*), que representa las probabilidades logarítmicas no normalizadas de la clase positiva.

- **Entrenamiento y optimización:** el modelo se entrena minimizando una función de costo llamada **entropía cruzada** (o *log loss*), que penaliza las predicciones que son seguras pero incorrectas. Dado que esta función es convexa, se puede utilizar el descenso de gradiente para encontrar los pesos óptimos de forma iterativa.
- **Evaluación mediante matriz de confusión:** es una tabla de $K \times K$ clases que desglosa el rendimiento del modelo comparando las etiquetas reales (filas) con las predichas (columnas). De aquí surgen cuatro valores críticos:
 - **Verdaderos Positivos** (TP, *True Positives*) y **Verdaderos Negativos** (TN, *True Negatives*): aciertos del modelo.
 - **Falsos Positivos** (FP, *False Positives*): error de tipo I o falsa alarma.
 - **Falsos Negativos** (FN, *False Negatives*): error de tipo II u omisión.

La Fig. 6.1 muestra la forma característica en “S” de la función sigmoide: valores de entrada muy negativos se aplastan hacia 0 y valores muy positivos hacia 1, con el punto de decisión en 0.5.

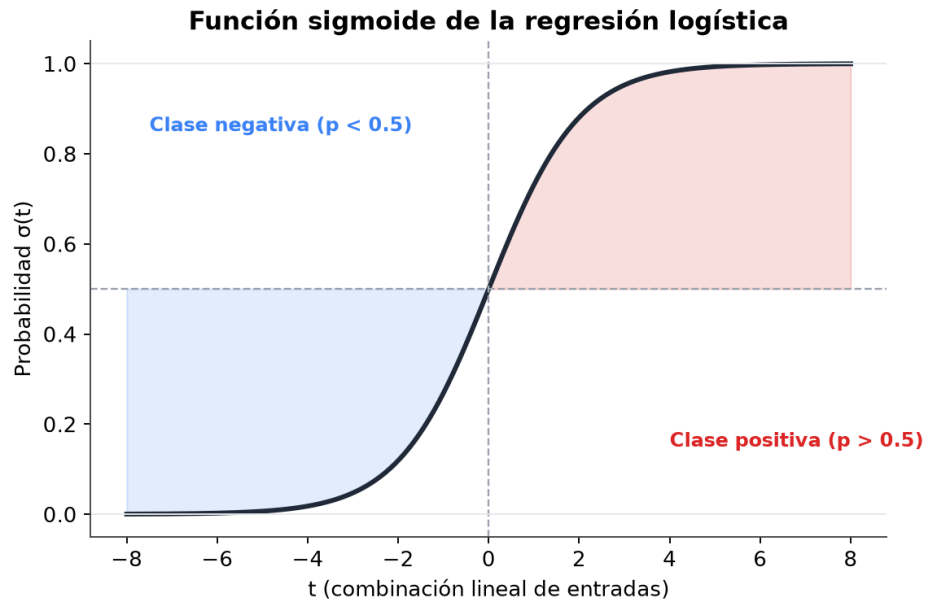


Figura6.1: Función sigmoide de la regresión logística.

La Fig. 6.2 muestra por qué es necesaria esta transformación: sobre el conjunto de datos *Default*, un ajuste por regresión lineal (izquierda) predice probabilidades absurdas fuera del rango $[0, 1]$, mientras que la regresión logística (derecha) siempre produce probabilidades válidas.

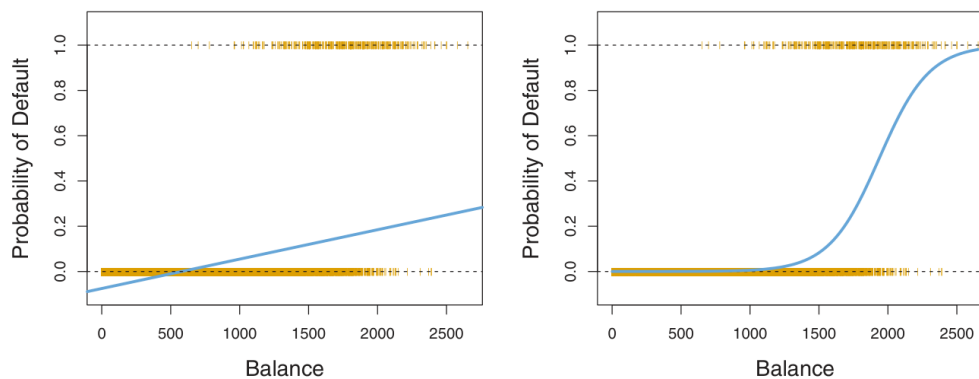


Figura6.2: Regresión lineal (izquierda) frente a regresión logística (derecha) sobre el conjunto de datos *Default*. Fuente: James, G., Witten, D., Hastie, T., & Tibshirani, R. (2013). *An Introduction to Statistical Learning*, Figura 4.2. Springer. Libro de libre distribución para uso educativo (statlearning.com).

6.3 Algoritmos clásicos de clasificación

Estos métodos estratifican el espacio de características o buscan fronteras de separación geométricas.

6.3.1 Árboles de decisión

Los árboles particionan el espacio de entrada de forma recursiva en “cajas” o rectángulos de alta dimensión.

- **Construcción:** siguen un enfoque de “divide y vencerás” mediante el algoritmo **CART**, que realiza divisiones binarias buscando maximizar la pureza de los nodos (reduciendo métricas de impureza como el índice Gini o la entropía).
- **Poda (pruning):** para evitar el sobreajuste, se utiliza la poda de complejidad de costos, que elimina ramas innecesarias basándose en pruebas estadísticas o conjuntos de validación.

La Fig. 6.3 ilustra cómo un árbol divide el espacio de características en regiones rectangulares mediante cortes sucesivos perpendiculares a los ejes.

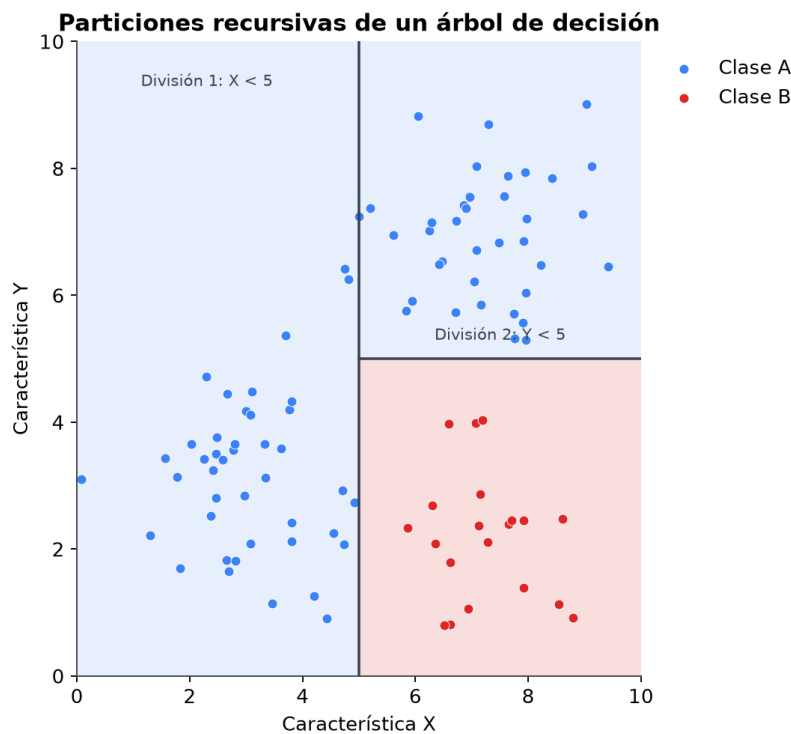


Figura6.3: Particiones recursivas de un árbol de decisión sobre dos características.

6.3.2 Bosques aleatorios (*random forests*)

Es un conjunto de árboles de decisión [Bre01] diseñado para reducir la varianza y mejorar la robustez. Existen dos técnicas para conformar el bosque:

- **Bagging**: utiliza el muestreo con reemplazo (*bootstrap*) para entrenar cada árbol con una versión diferente de los datos.
- **Decorrelación**: para asegurar que los árboles sean diversos, en cada división solo se considera un subconjunto aleatorio de características.

La Fig. 6.4 esquematiza el proceso completo: a partir del *dataset* original se generan varias muestras *bootstrap*, cada una entrena un árbol distinto (usando además un subconjunto aleatorio de *features* en cada división) y las predicciones individuales se combinan mediante votación mayoritaria (clasificación) o promedio (regresión).

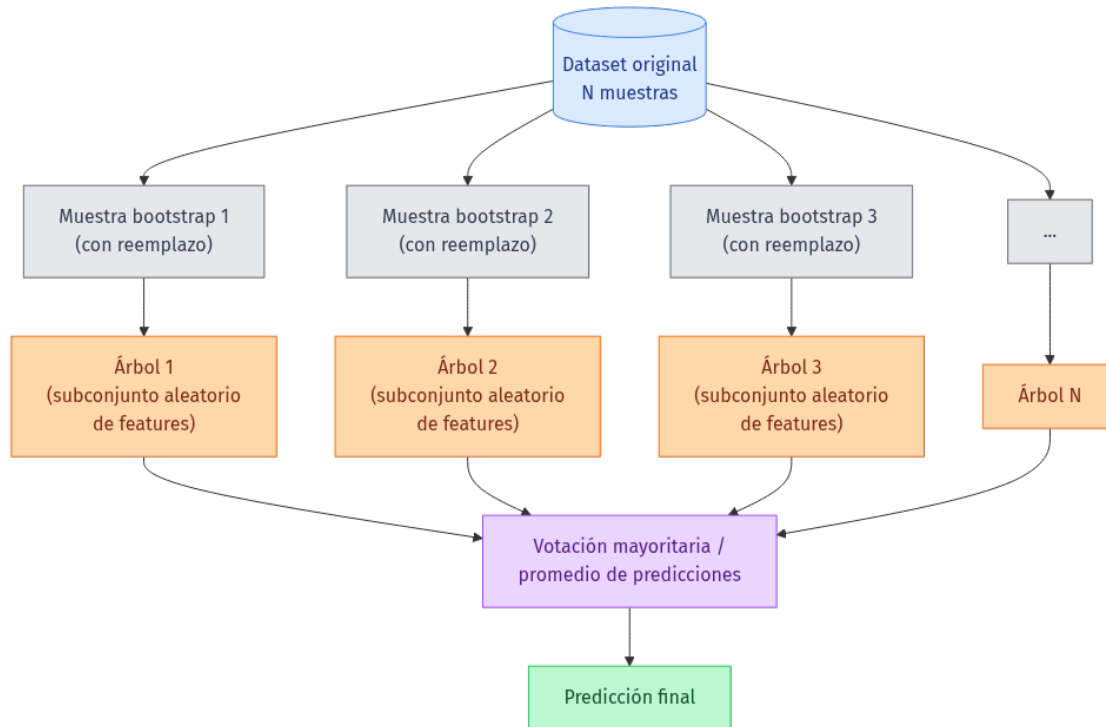


Figura6.4: Bagging: cada árbol se entrena con una muestra *bootstrap* distinta y las predicciones se combinan por votación.

6.3.3 Máquinas de vectores de soporte (SVM, *Support Vector Machines*)

Este modelo [CV95] busca encontrar un **hiperplano** que separe las clases con el **margen máximo** posible.

- **Vectores de soporte**: la frontera de decisión está determinada únicamente por las muestras más cercanas al hiperplano; mover otros puntos no altera el modelo.
- **Margen blando y truco del *kernel***: las SVM manejan datos no separables linealmente mediante variables de holgura (*slack variables*) y el truco del *kernel*, que mapea implícitamente los datos a espacios de mayor dimensión para encontrar separaciones complejas.

La Fig. 6.5 muestra el hiperplano óptimo (línea continua), el margen máximo (banda gris) y los vectores de soporte (puntos rodeados en verde), que son los únicos que determinan la posición de la frontera.

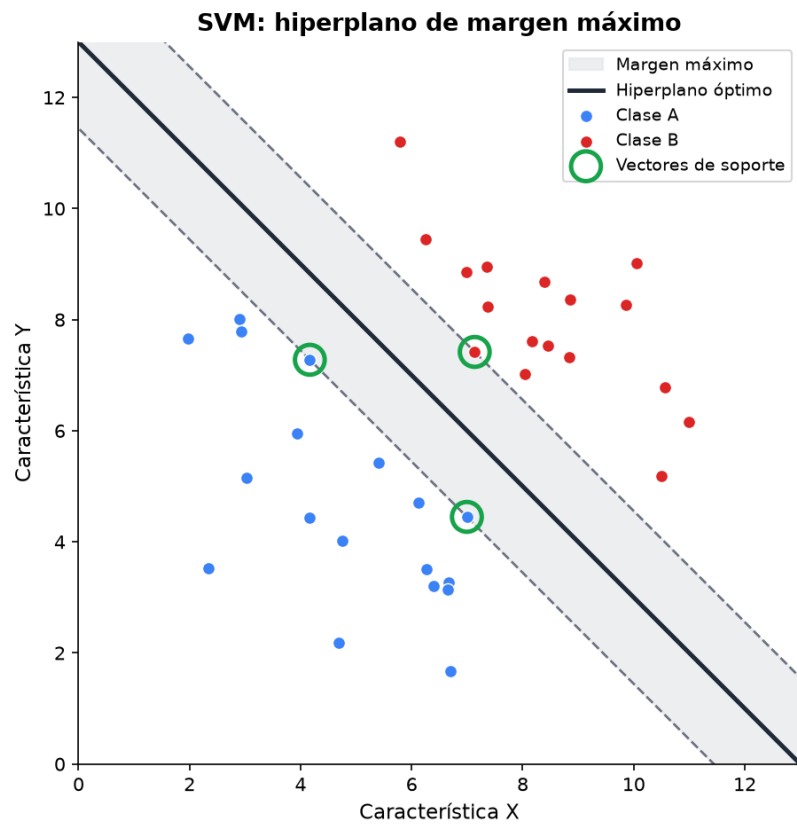


Figura6.5: SVM: hiperplano de margen máximo y vectores de soporte.

6.4 Métodos de ensamble avanzados

Combinan múltiples modelos para obtener una predicción superior a la de cualquier modelo individual.

- **Voting y stacking**: el voto mayoritario (duro) o el promedio de probabilidades (suave) combina modelos independientes. El *stacking* entrena un meta-modelo que aprende a combinar las salidas de los modelos base.

La Fig. 6.6 compara ambos enfoques: en *voting*, los modelos se entrenan de forma independiente y sus salidas se combinan con una regla fija (voto o promedio); en *stacking*, las predicciones de los modelos base se convierten en la entrada de un meta-modelo que aprende cómo combinarlas.

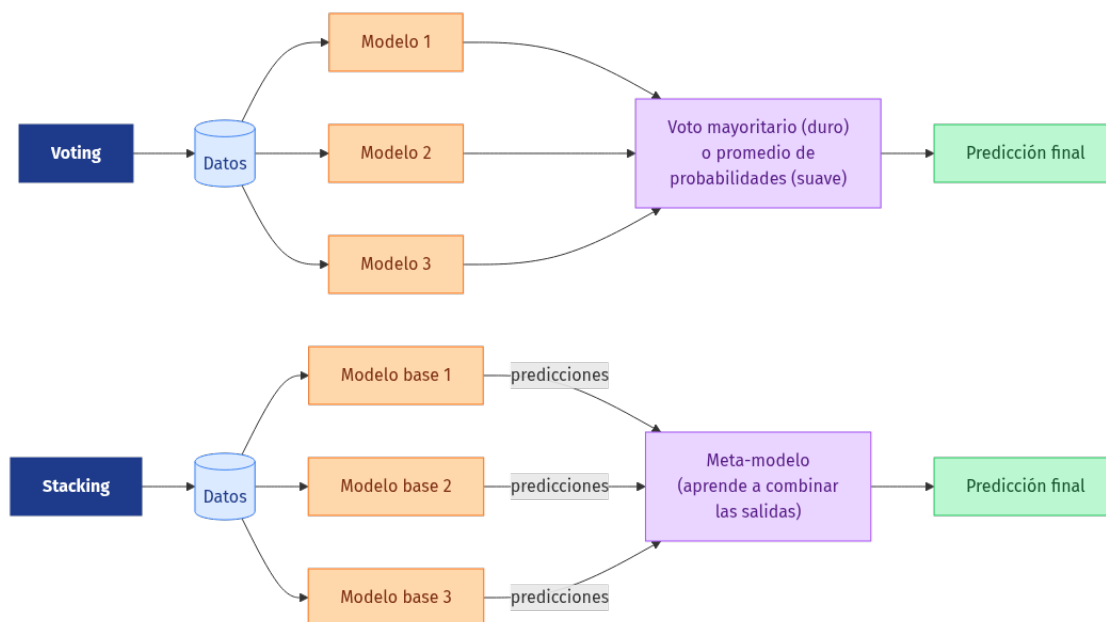


Figura6.6: *Voting* frente a *stacking*: regla fija de combinación frente a meta-modelo que aprende a combinar.

- **Boosting** [Fri01]: a diferencia del entrenamiento en paralelo de los bosques aleatorios, el *boosting* entrena modelos de forma secuencial, donde cada nuevo predictor intenta corregir los errores cometidos por sus predecesores. Ejemplos modernos incluyen *XGBoost* y *Gradient Boosting*.

La Fig. 6.7 ilustra el entrenamiento secuencial del *boosting*: cada modelo débil se entrena sobre los datos ponderados según los errores del modelo anterior, dando más peso a las instancias mal clasificadas, y la predicción final combina todos los modelos mediante una suma ponderada.



Figura6.7: *Boosting*: los modelos se entrenan secuencialmente, cada uno corrigiendo los errores del anterior.

6.5 Introducción a las redes neuronales

Representan el aprendizaje de representaciones sucesivas a través de capas de filtrado.

- **Arquitectura:** se componen de una capa de entrada, múltiples capas ocultas (densamente conectadas) y una capa de salida.
- **Funciones de activación:** introducen no linealidad para aprender patrones complejos. **ReLU** es el estándar para capas ocultas, mientras que **Softmax** se usa en la capa de salida para clasificaciones de múltiples clases.
- **Retropropagación** (*backpropagation*) [RHW86]: es el mecanismo central de aprendizaje; utiliza la regla de la cadena del cálculo para propagar el error desde la salida hacia atrás, ajustando los pesos de la red para reducir la pérdida total.

La Fig. 6.8 esquematiza una red totalmente conectada con una capa de entrada, una capa oculta y una capa de salida: cada conexión representa un peso que se ajusta durante el entrenamiento.

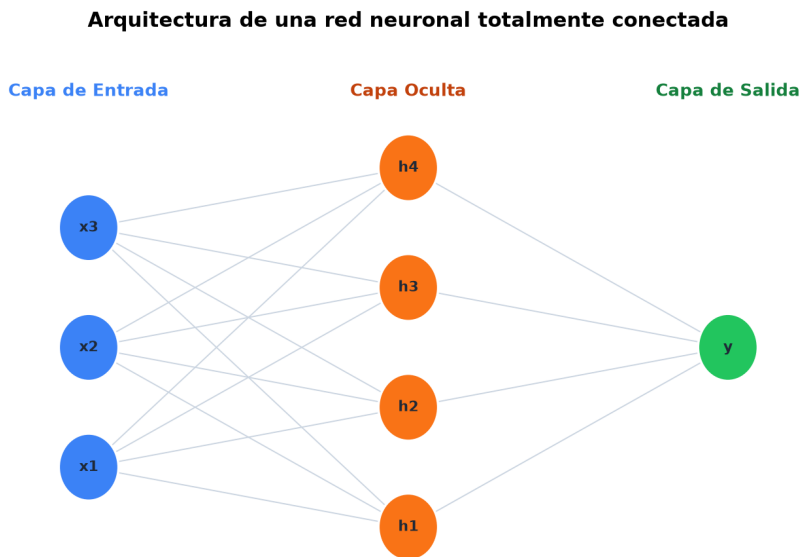


Figura6.8: Arquitectura de una red neuronal totalmente conectada.

6.6 Evalaución del rendimeinto en clasificación

La evaluación es el proceso fundamental para medir la capacidad de generalización de un modelo, es decir, su aptitud para realizar predicciones precisas sobre datos que nunca ha visto anteriormente. Un modelo que rinde excepcionalmente en los datos de entrenamiento pero falla en datos nuevos está sufriendo de sobreajuste (*overfitting*) y carece de utilidad práctica.

6.6.1 Protocolos de validación y división de datos

Para medir la generalización de forma fiable, es obligatorio dividir el conjunto de datos disponible en particiones independientes.

- **Entrenamiento** (*train*), **validación** (*validation*) y **prueba** (*test*): en situaciones de abundancia de datos, se reserva un conjunto de entrenamiento para ajustar los pesos del modelo, uno de validación para seleccionar la mejor arquitectura o hiperparámetros, y uno de prueba que se mantiene en una “bóveda” para una evaluación final e imparcial.
- **Validación cruzada de K pliegues** (*K-fold Cross-Validation*): es la estrategia recomendada cuando los datos son escasos. Consiste en dividir los datos en K particiones; el modelo se entrena K veces, usando cada vez una partición distinta para validación y las $K - 1$ restantes para entrenamiento. El resultado final es el promedio de las puntuaciones obtenidas en los K experimentos, lo que reduce la varianza de la estimación.
- **Estratificación**: en tareas de clasificación, es vital asegurar que cada partición mantenga la proporción original de las clases, especialmente cuando estas están desbalanceadas.

6.6.2 Métricas de éxito para clasificadores

La elección de la métrica debe alinearse con el objetivo del negocio y la naturaleza de los datos.

La matriz de confusión

Es una herramienta bidimensional donde las filas representan las clases reales y las columnas las predichas. Permite identificar cuatro valores esenciales:

- **Verdaderos Positivos** (TP, *True Positives*): instancias positivas correctamente clasificadas.
- **Verdaderos Negativos** (TN, *True Negatives*): instancias negativas correctamente clasificadas.
- **Falsos Positivos** (FP, *False Positives*): error de tipo I o “falsa alarma”.
- **Falsos Negativos** (FN, *False Negatives*): error de tipo II u “omisión”.

Métricas derivadas

- **Exactitud** (*accuracy*): fracción de predicciones correctas sobre el total. Puede ser engañosa en conjuntos de datos sesgados (*skewed datasets*); por ejemplo, si el 90% de las muestras pertenecen a una clase, un modelo que siempre prediga esa clase tendrá un 90% de exactitud sin haber aprendido nada.
- **Precisión** (*precision*): capacidad del modelo para no etiquetar como positiva una muestra que es negativa: $\frac{TP}{TP+FP}$
- **Sensibilidad** (*recall*): capacidad de encontrar todas las muestras positivas reales: $\frac{TP}{TP+FN}$
- **Puntuación F1** (*F1-score*): media armónica de precisión y sensibilidad, útil cuando se busca un equilibrio entre ambas.

- **ROC AUC:** el área bajo la curva de la característica operativa del receptor mide la probabilidad de que el modelo clasifique una muestra positiva aleatoria por encima de una negativa. Un valor de 1.0 es perfecto y 0.5 equivale a una clasificación aleatoria.

La Fig. 6.9 compara un buen clasificador (curva alejada de la diagonal, AUC alto) frente al comportamiento de un clasificador aleatorio (línea diagonal, AUC = 0.5).

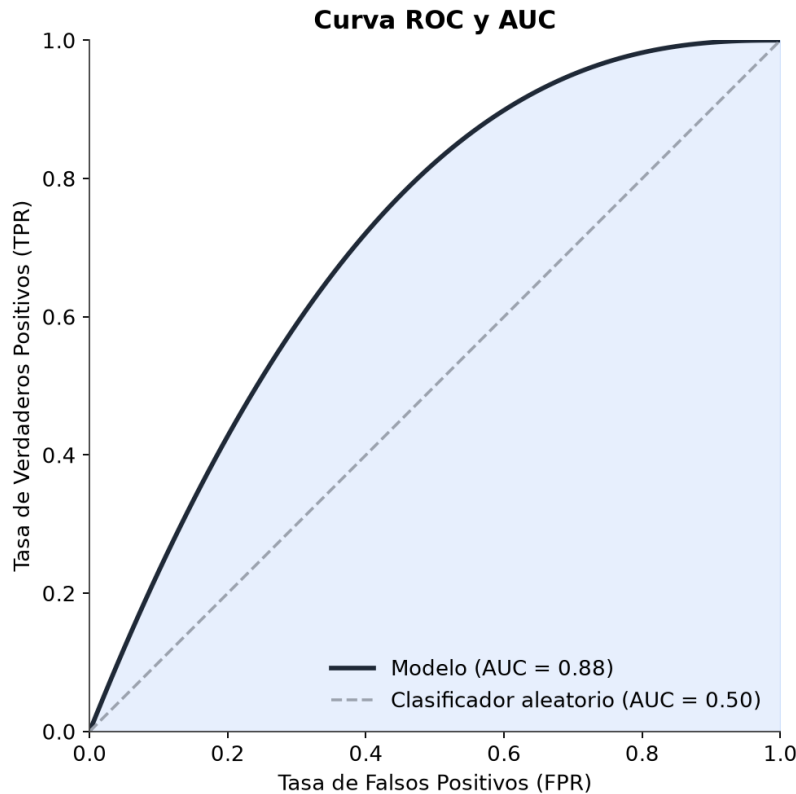


Figura 6.9: Curva ROC y área bajo la curva (AUC).

6.6.3 Estrategias de entrenamiento y diagnóstico

La evaluación no solo ocurre al final, sino que debe guiar todo el proceso de desarrollo.

- **Superar una línea de base (*baseline*):** antes de iniciar con modelos complejos, se debe establecer una línea de base trivial (como un clasificador aleatorio). Si el modelo no puede superar este umbral de sentido común, es probable que los datos no contengan información suficiente o el enfoque sea erróneo.
- **Monitoreo de curvas de aprendizaje:** graficar la pérdida (*loss*) y la exactitud tanto del entrenamiento como de la validación permite detectar el punto exacto de sobreajuste (cuando la pérdida de entrenamiento baja pero la de validación empieza a subir).
- **Parada temprana (*early stopping*):** estrategia que utiliza un *callback* para interrumpir el entrenamiento automáticamente cuando la métrica de validación deja de mejorar, ahorrando tiempo y evitando el sobreajuste.
- **Análisis de errores:** inspeccionar manualmente las muestras donde el modelo falla ayuda a entender qué patrones específicos está confundiendo el sistema (por ejemplo, confundir un “3” con un “5” en reconocimiento de dígitos).
- **A/B testing:** tras el despliegue, se recomienda realizar pruebas aleatorizadas para medir el impacto real del modelo en comparación con el proceso anterior.

6.7 Ejemplo práctico en Java con SMILE

El repositorio `programacion-avanzada-smile` incluye el ejemplo completo `Ejemplo03Clasificacion.java`, que entrena varios clasificadores sobre el conjunto de datos *Iris* con la librería **SMILE**. El siguiente fragmento resume sus pasos principales.

SMILE ofrece dos estilos de API: **con fórmula** (`fit(Formula, DataFrame)`), usado por los modelos basados en árboles, que aceptan variables categóricas sin codificar) y **con arrays** (`fit(double[][] x, int[] y)`), usado por KNN, SVM, regresión logística o redes neuronales).

```
import smile.classification.KNN;
import smile.classification.LogisticRegression;
import smile.classification.RandomForest;
import smile.data.DataFrame;
import smile.data.formula.Formula;
import smile.io.Read;
import smile.math.MathEx;
import smile.validation.CrossValidation;
import smile.validation.metric.Accuracy;
import smile.validation.metric.ConfusionMatrix;

MathEx.setSeed(42); // semilla global: particiones y modelos reproducibles

DataFrame iris = Read.csv("data/iris.csv", "header=true").factorize("species");
Formula formula = Formula.lhs("species"); // species ~ resto de columnas

Utiles.Particion particion = Utiles.split(iris, 0.7);
DataFrame train = particion.train();
DataFrame test = particion.test();

// 1. Random Forest (API con formula): trabaja directamente sobre el DataFrame
RandomForest bosque = RandomForest.fit(formula, train, new RandomForest.Options(200));

int[] yTest = formula.y(test).toIntArray();
int[] prediccion = new int[test.nrow()];
for (int i = 0; i < test.nrow(); i++) {
    prediccion[i] = bosque.predict(test.get(i));
}
System.out.printf("Accuracy = %.2f%%\n", 100.0 * Accuracy.of(yTest, prediccion));
System.out.println(ConfusionMatrix.of(yTest, prediccion));

// 2. Modelos que trabajan con arrays: KNN y regresion logistica
double[][] xTrain = formula.x(train).toArray();
int[] yTrain = formula.y(train).toIntArray();
double[][] xTest = formula.x(test).toArray();

KNN<double[]> knn = KNN.fit(xTrain, yTrain, 5);
LogisticRegression logistica = LogisticRegression.fit(xTrain, yTrain,
    new LogisticRegression.Options(0.1, 1E-5, 500)); // lambda, tolerancia,
    iteraciones
System.out.printf("KNN (k=5) accuracy = %.2f%%\n",
    100.0 * Accuracy.of(yTest, knn.predict(xTest)));
System.out.printf("Regresion logistica accuracy = %.2f%%\n",
    100.0 * Accuracy.of(yTest, logistica.predict(xTest)));

// 3. Validacion cruzada de 10 pliegues: la estimacion honesta del rendimiento
var cv = CrossValidation.classification(10, formula, iris,
```

(continues on next page)

(continued from previous page)

```
(f, datos) -> RandomForest.fit(f, datos, new RandomForest.Options(200));
System.out.printf("Accuracy media = %.4f (desv. %.4f)%n",
    cv.avg().accuracy(), cv.std().accuracy());
```

Algunas ideas que conviene observar al ejecutarlo:

- `factorize("species")` convierte la columna de texto con la especie en una variable categórica con códigos enteros, que es lo que esperan los clasificadores.
- `ConfusionMatrix.of` construye la matriz de confusión vista en esta sección, y `Accuracy.of` calcula la exactitud a partir de las etiquetas reales y las predichas.
- El primer parámetro de `LogisticRegression.Options` es λ , la fuerza de la regularización L_2 aplicada a los pesos del modelo.
- El ejemplo completo muestra además la predicción «blanda» (probabilidades a posteriori de cada clase), la persistencia del modelo en disco y el uso de *encoders* (`LEVEL`, `ONE_HOT`) para variables categóricas con un segundo conjunto de datos.

Para ejecutarlo desde la raíz del repositorio: `mvn exec:java -Dexec.mainClass=es.usal.smile.Ejemplo03Clasificacion`.

6.8 Resumen

- **Regresión logística:** extensión de regresión lineal para clasificación.
- **Matriz de confusión:** visualiza el desempeño.
- **Métricas:** elige según lo que importa (precisión o recall).
- **Árboles de decisión:** interpretables pero proclives a overfitting.
- **Random Forest:** ensemble que reduce overfitting.
- **SVM:** encuentra separación óptima con kernel trick.
- **Ensamblados:** combinan varios algoritmos de clasificación.
- **Redes neuronales:** poderosas pero complejas.

Siguiente: en la Sección 3 exploraremos aprendizaje no supervisado, donde trabajamos con datos sin etiquetas.

Part III

Sección 3 - Aprendizaje no supervisado

SECCIÓN 3: APRENDIZAJE NO SUPERVISADO

Bienvenido a la tercera y última sección principal del curso. Aquí exploraremos técnicas para extraer patrones de datos **sin etiquetas**.

7.1 ¿Qué es aprendizaje no supervisado?

A diferencia de los bloques anteriores enfocados en el aprendizaje supervisado, donde el entrenamiento está guiado por una variable objetivo clara o etiqueta (*label*), el aprendizaje no supervisado representa la desafiante tarea de enfrentarse a conjuntos de datos que carecen de cualquier tipo de salida esperada o anotación. En este paradigma, únicamente disponemos de una colección de características de entrada X (o variables predictoras), pero no contamos con un “maestro” o supervisor que proporcione las respuestas correctas para guiar el proceso de aprendizaje.

Técnicamente, mientras que el aprendizaje supervisado busca aproximar una función de mapeo directo entre entradas y salidas o estimar una **distribución condicional** $p(y|x)$, el aprendizaje no supervisado trabaja “a ciegas” para descubrir la **estructura latente**, las asociaciones estadísticas intrínsecas o la **distribución incondicional** de los propios datos $p(x)$. Como célebremente ilustró el científico de la computación Yann LeCun: “Si la inteligencia fuera un pastel, el aprendizaje no supervisado sería el cuerpo del pastel (el bizcocho), el aprendizaje supervisado la cobertura (el glaseado), y el aprendizaje por refuerzo la cereza”. Esta analogía subraya que la inmensa mayoría de la información disponible en el mundo real no viene etiquetada, haciendo que este paradigma sea el pilar fundamental sobre el cual se erige el verdadero entendimiento de los datos.

7.1.1 Tipos principales de aprendizaje no supervisado

1. **Clustering** (agrupamiento): su objetivo es agrupar datos similares.
2. **Reducción de dimensionalidad**: simplifica datos complejos.
3. **Detección de anomalías**: este tipo permite identificar casos raros o inusuales.

7.2 Ejemplos y utilidades prácticas

El aprendizaje no supervisado no busca realizar predicciones cuantitativas o cualitativas puntuales. Su valor reside en extraer conocimiento, simplificar estructuras y revelar patrones ocultos para que sean interpretables por el ser humano o sirvan de preparación para otros modelos:

- **Segmentación de mercado** (*market segmentation*): al analizar grandes volúmenes de datos demográficos y transaccionales de consumidores (como ingresos, ocupación o hábitos de gasto), las empresas pueden agrupar a las personas en nichos homogéneos para dirigir campañas publicitarias específicas o desarrollar productos personalizados sin necesidad de definir estas categorías de antemano.

- **Visualización de datos complejos:** proyectar datasets de altísima dimensionalidad (con cientos de atributos) a representaciones legibles de dos o tres dimensiones, preservando la estructura y las relaciones geométricas para que un analista pueda comprender de forma intuitiva cómo se organizan los datos.
- **Sistemas de recomendación y filtrado colaborativo:** agrupar usuarios con gustos afines o productos con características similares para predecir preferencias implícitas basándose en comportamientos de consumo históricos.
- **Detección de anomalías:** modelar el comportamiento estadístico de los datos “normales” para identificar de manera automática aquellas instancias sospechosas que se desvían significativamente del patrón general (por ejemplo, para detectar fraudes financieros o fallos en líneas de manufactura).

7.3 ¿Qué encontrarás en esta sección?

- **Clustering:**
 - *K-means*: agrupamiento basado en centroides, inicialización (*K-means++*) y elección de *K* con el método del codo.
 - *Clustering* jerárquico: enfoque aglomerativo, dendrogramas y criterios de enlace.
 - DBSCAN: agrupamiento basado en densidad, capaz de detectar formas arbitrarias y ruido.
 - Técnicas de evaluación: coeficiente de silueta, métricas internas, criterios de teoría de la información y evaluación externa con datos etiquetados.
- **Reducción de dimensionalidad:**
 - La maldición de la dimensionalidad: por qué trabajar en espacios de muchas variables es un problema.
 - PCA: proyección lineal que maximiza la varianza retenida, SVD y reconstrucción.
 - Aprendizaje de variedades (*manifold learning*): la hipótesis de la variedad y el ejemplo del *Swiss roll*.
 - Métodos no lineales: *Kernel PCA*, *t-SNE* y UMAP.
- **Detección de anomalías:**
 - *Anomaly detection* frente a *novelty detection*: la diferencia según la contaminación de los datos de entrenamiento.
 - Enfoques estadísticos y de reconstrucción: mezclas gaussianas (GMM) y error de reconstrucción con PCA.
 - *Isolation forest*: aislar las observaciones mediante particiones aleatorias.
 - *Local Outlier Factor* (LOF): detectar anomalías locales según la densidad del vecindario.
 - *One-class SVM*: delimitar la región de comportamiento normal.

7.4 Nota importante

El aprendizaje no supervisado es más **arte que ciencia**. No hay una única respuesta “correcta”. Tu interpretación del dominio y el *feedback* externo son cruciales.

Empecemos: dirígete al Capítulo 3.1 para aprender sobre *clustering*, el enfoque no supervisado más popular.

3.1 CLUSTERING

8.1 Introducción

El aprendizaje no supervisado representa un paradigma fundamental de la inteligencia artificial, caracterizado por la ausencia de etiquetas o variables objetivo que guíen el proceso de entrenamiento. En este bloque, el sistema no recibe un criterio de “verdad fundamental” (*ground-truth*), sino que debe explorar las relaciones matemáticas intrínsecas de los datos para revelar su estructura oculta.

Dentro de este paradigma, el **clustering** (o agrupamiento) es la técnica más extendida, orientada a particionar un conjunto de datos en subgrupos cuyos miembros compartan una alta similitud interna y, al mismo tiempo, presenten una alta diferenciación respecto a los miembros de otros subgrupos.

8.2 Algoritmos de *clustering*

Existen diferentes algoritmos de **clustering**. A continuación, se explican los mas relevantes actualmente.

8.2.1 Algoritmo *K-means*

El *K-means* [Mac67] es un algoritmo de agrupación que realiza las particiones basada en centroides. El algoritmo es una técnica de agrupamiento iterativo que busca particionar un conjunto de observaciones en K *clusters* distintos. Es un enfoque geométrico basado en el concepto de centroides, que actúan como los centros de gravedad de cada grupo.

La Fig. 8.1 resume el ciclo del algoritmo: tras asignar cada muestra a su centroide y calcular la inercia (WCSS), se actualizan los centroides y el proceso se repite hasta alcanzar la convergencia.

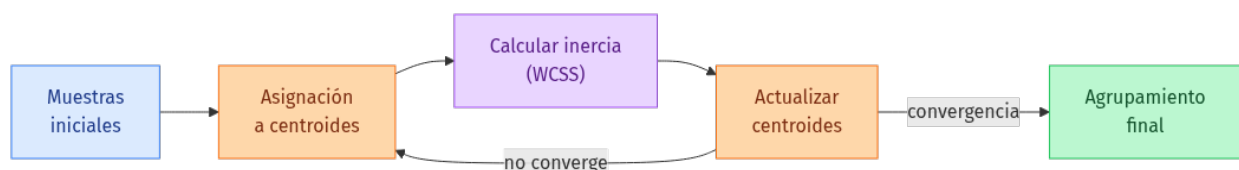


Figura8.1: Ciclo iterativo de *K-means*.

Mecánica algorítmica y convergencia

La meta matemática de *K-means* es minimizar la inercia intracluster, formalmente conocida como la suma de los cuadrados dentro del *cluster* (WCSS - *Within-cluster Sum of Squares*). El proceso de optimización se divide en los siguientes pasos iterativos:

1. **Inicialización:** se definen K puntos en el espacio de características para que actúen como centroides iniciales.
2. **Paso de asignación** (expectación): cada muestra del *dataset* se asigna al centroide más cercano utilizando una métrica de distancia, típicamente la distancia Euclidiana: $d(x, y) = \|x - y\|_2 = \sqrt{\sum_{i=1}^D (x_i - y_i)^2}$
3. **Paso de actualización** (maximización): se recalcula la posición de cada centroide como el promedio matemático (la media aritmética) de todas las muestras asignadas a dicho cluster.
4. **Convergencia:** los pasos 2 y 3 se repiten de forma iterativa hasta que los centroides no cambien su posición significativamente o las asignaciones de las muestras permanezcan estables.

La Fig. 8.2 muestra las tres etapas sobre un conjunto de datos con tres grupos naturales: los centroides (marcados con X) parten de posiciones aleatorias y convergen hacia el centro de cada grupo tras pocas iteraciones.

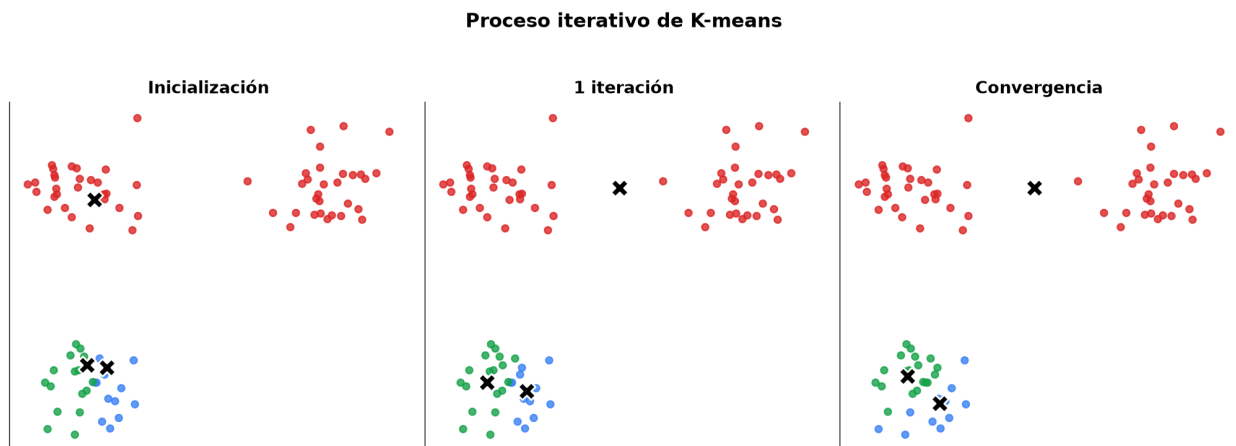


Figura8.2: Proceso iterativo de *K-means*: de la inicialización a la convergencia.

El problema de los mínimos locales e inicialización (*K-means++*)

K-means es un algoritmo heurístico y, por tanto, es altamente sensible a la inicialización aleatoria de sus centroides. Si los centroides iniciales se colocan en regiones desfavorables del espacio de características, el algoritmo puede converger en mínimos locales subóptimos.

Para mitigar esta vulnerabilidad, se utiliza la inicialización *K-means++*:

- El primer centroide se selecciona de manera uniformemente aleatoria entre los puntos de datos.
- Los centroides subsiguientes se eligen de forma probabilística, donde la probabilidad de seleccionar un punto como nuevo centroide es proporcional al cuadrado de la distancia al centroide más cercano ya existente: $P(x) \propto D(x)^2$

Este método de siembra inteligente asegura que los centroides iniciales estén ampliamente distribuidos por todo el espacio de características, acelerando la convergencia y mejorando la calidad del agrupamiento final

Selección de la complejidad del modelo

Método del codo (*elbow method*)

Dado que el número de *clusters* K es un hiperparámetro que debe ser definido por el usuario antes del entrenamiento, se requiere un protocolo científico para su selección.

El método del codo consiste en graficar el valor del WCSS en función de diferentes valores de K :

- A medida que K aumenta, la WCSS disminuye de manera natural porque los *clusters* se vuelven más pequeños y los puntos están más cerca de sus respectivos centroides.
- El valor óptimo de K se localiza en el punto de inflexión de la curva (el “codo”), donde el descenso de la inercia deja de ser abrupto y se vuelve lineal. Esto representa un equilibrio óptimo entre la complejidad del modelo y la cohesión de los grupos.

La Fig. 8.3 muestra un ejemplo típico: la inercia cae bruscamente hasta $K = 3$ y después apenas mejora, señalando que 3 es el número de clusters más razonable para estos datos.

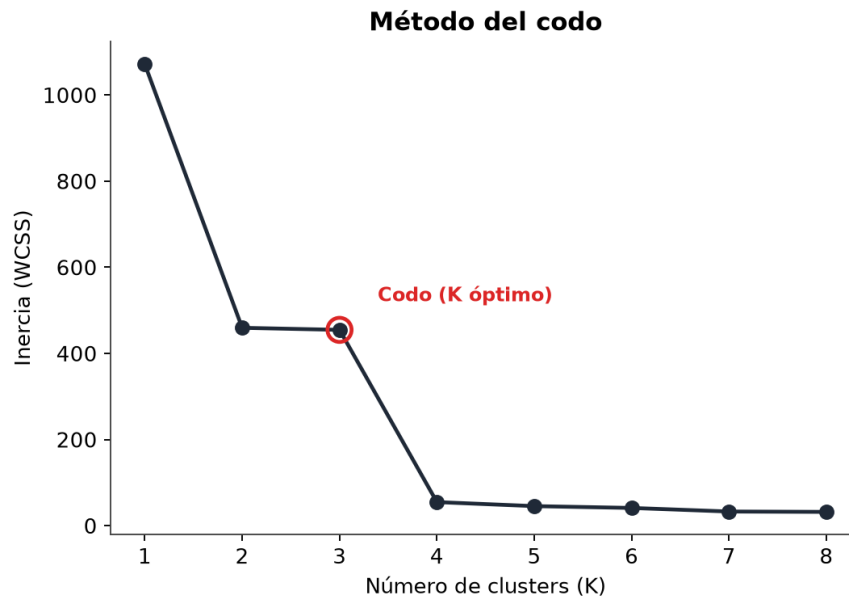


Figura8.3: Método del codo: la inercia deja de disminuir bruscamente a partir de $K = 3$.

8.2.2 Clustering jerárquico: aglomerativo y dendrogramas

El *clustering* jerárquico agrupa datos sin la necesidad de definir de antemano el número de grupos K . Su principal ventaja es que genera una estructura jerárquica de agrupamiento que puede visualizarse fácilmente.

Enfoque aglomerativo (*bottom-up*)

El método más común es el jerárquico aglomerativo, que opera de abajo hacia arriba. El algoritmo comienza tratando a cada observación individual como un único *cluster* independiente. En cada paso iterativo, se calculan las distancias entre todos los *clusters* y se fusionan los dos grupos más cercanos. Este proceso de fusión se repite de manera sucesiva hasta que todas las muestras quedan integradas en un único *cluster* global.

Dendrogramas e interpretación de cortes

La jerarquía de fusiones se representa gráficamente mediante un diagrama de árbol llamado dendrograma. El eje vertical del dendrograma representa la distancia de fusión (o disimilitud copenhética) entre los subgrupos.

El eje horizontal organiza las observaciones individuales de forma que las ramas similares queden adyacentes. El usuario puede determinar el número final de *clusters* realizando un corte horizontal en el dendrograma a una altura de disimilitud específica. La cantidad de líneas verticales interceptadas por el corte define el número de grupos resultantes, ofreciendo una flexibilidad interpretativa que algoritmos rígidos como K -means no poseen.

La Fig. 8.4 muestra un dendrograma real generado sobre datos con tres grupos: la línea roja discontinua marca un corte que produce exactamente 3 clusters (uno por color).

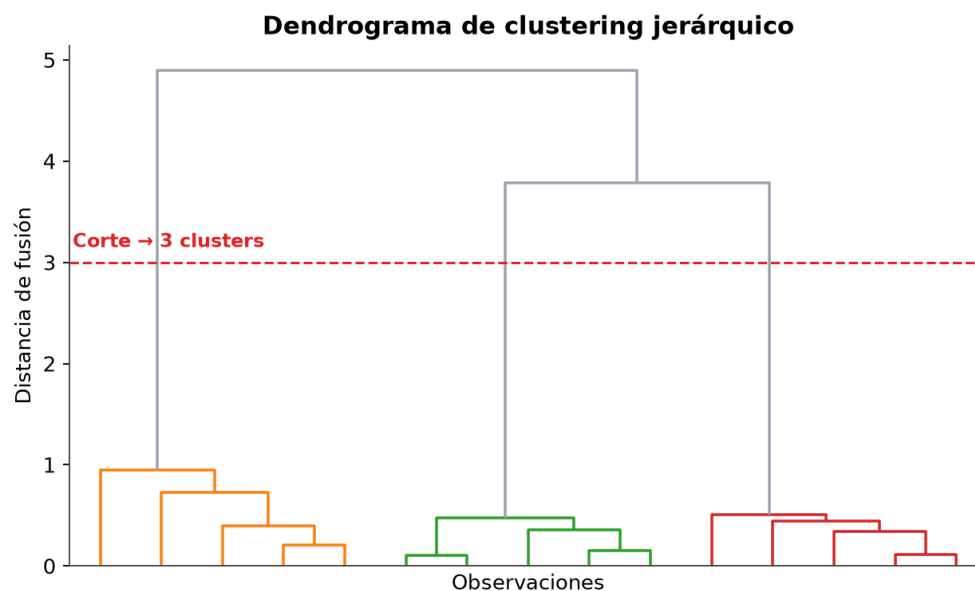


Figura 8.4: Dendrograma de *clustering* jerárquico con un corte que produce 3 clusters.

Criterios de enlace (*linkage methods*)

Para determinar qué *clusters* deben fusionarse en cada paso, se debe definir cómo medir la distancia o disimilitud entre grupos que contienen múltiples observaciones. Los principales métodos de enlace son:

- **Complete linkage** (enlace completo): calcula la distancia máxima entre cualquier miembro del primer *cluster* y cualquier miembro del segundo *cluster*: $D(A, B) = \max_{x \in A, y \in B} d(x, y)$ Produce dendrogramas compactos y *clusters* de formas esféricas bien balanceadas.
- **Single linkage** (enlace simple): calcula la distancia mínima entre cualquier miembro del primer *cluster* y cualquier miembro del segundo *cluster*: $D(A, B) = \min_{x \in A, y \in B} d(x, y)$ Es sensible al ruido y puede generar el efecto de encadenamiento (*chaining*), donde los *clusters* se fusionan de forma alargada y difusa.
- **Average linkage** (enlace promedio): calcula el promedio de todas las distancias entre cada punto del primer *cluster* y cada punto del segundo: $D(A, B) = \frac{1}{|A||B|} \sum_{x \in A} \sum_{y \in B} d(x, y)$ Es un criterio robusto que equilibra la estabilidad del enlace completo con la tolerancia del enlace simple

La Fig. 8.5 compara los tres criterios de enlace sobre datos reales de expresión génica del conjunto *NCI60* (líneas celulares de cáncer): nótese cómo el enlace simple (abajo) produce el efecto de encadenamiento característico, mientras que el enlace completo y el promedio (arriba y centro) generan agrupamientos más equilibrados.

8.2.3 DBSCAN: agrupamiento basado en densidad

El algoritmo DBSCAN (*Density-Based Spatial clustering of Applications with Noise*) [EKSX96] ofrece un enfoque radicalmente diferente a *K-means* y al **clustering** jerárquico al definir los grupos en función de la densidad local de los datos en el espacio de características. Esto le permite descubrir *clusters* de formas geométricas arbitrarias y aislar de forma natural las muestras de ruido

Fundamentos y parámetros críticos

DBSCAN requiere la configuración de dos hiperparámetros esenciales que definen la densidad local del entorno:

- **ϵ (Epsilon)**: el radio de vecindad que define el entorno alrededor de cualquier punto.
- **MinPts** (muestras mínimas): el número mínimo de puntos que deben existir dentro del radio ϵ para considerar que esa región es densa.

Clasificación de puntos en DBSCAN

Durante su ejecución, DBSCAN examina cada punto del *dataset* y lo clasifica en una de tres categorías exclusivas según las condiciones de densidad de su vecindad:

- **Core points** (puntos núcleo): un punto es catalogado como núcleo si su vecindad de radio ϵ contiene al menos un número de muestras igual o superior a MinPts.
- **Border points** (puntos frontera): son aquellos puntos que no cumplen con el requisito de densidad mínima de MinPts para ser considerados núcleo, pero residen dentro de la vecindad ϵ de algún punto que sí es núcleo.
- **Noise points** (puntos de ruido / *outliers*): cualquier punto que no es clasificado como núcleo ni como frontera. Estos puntos se consideran anomalías o valores atípicos y no son asignados a ningún *cluster*.

La Fig. 8.6 ilustra los tres tipos con $\epsilon = 1$ y MinPts = 4: los puntos azules tienen al menos 4 vecinos dentro de su círculo (núcleo); los naranjas caen dentro del círculo de un punto núcleo pero no alcanzan MinPts por sí mismos (frontera); y la cruz roja no tiene ningún punto núcleo cerca (ruido).

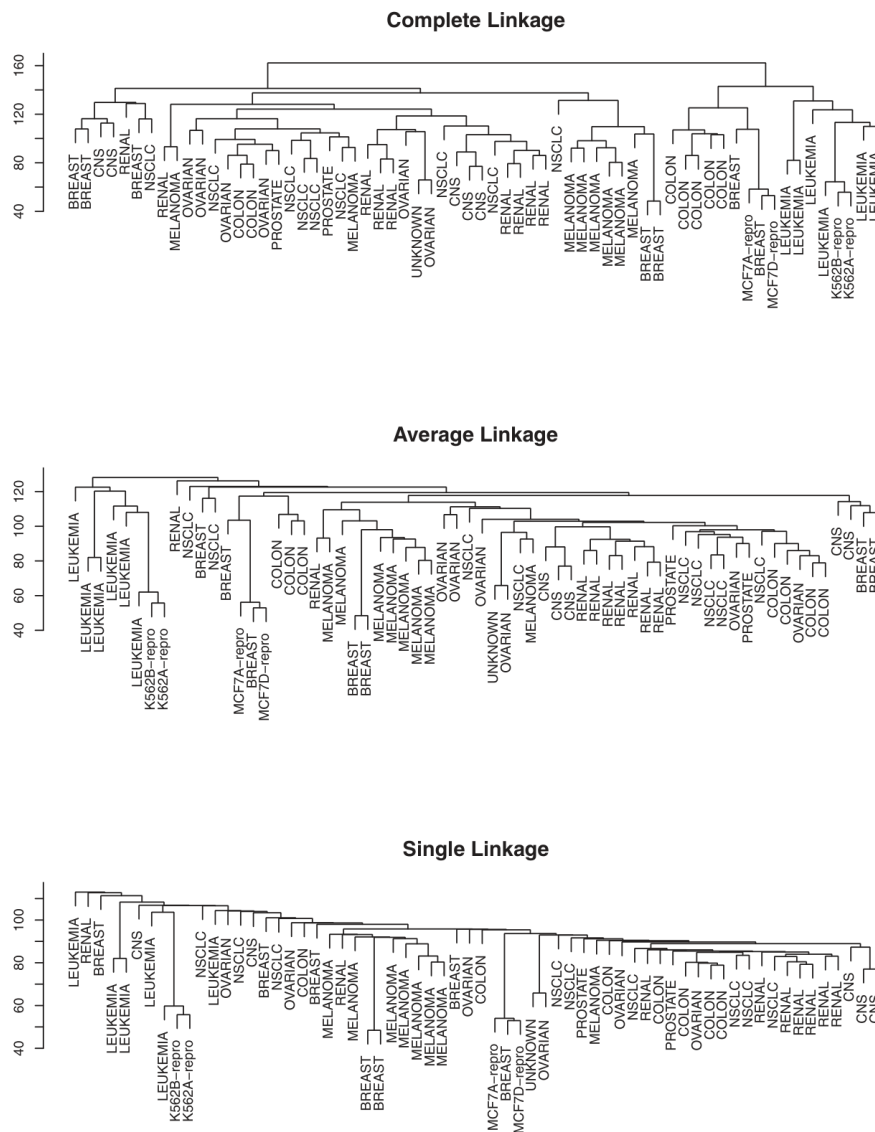


Figura 8.5: Clustering jerárquico del conjunto de datos *NCI60* con enlace completo, promedio y simple. Fuente: James, G., Witten, D., Hastie, T., & Tibshirani, R. (2013). *An Introduction to Statistical Learning*, Figura 10.17. Springer. Libro de libre distribución para uso educativo (statlearning.com).

Tipos de puntos en DBSCAN

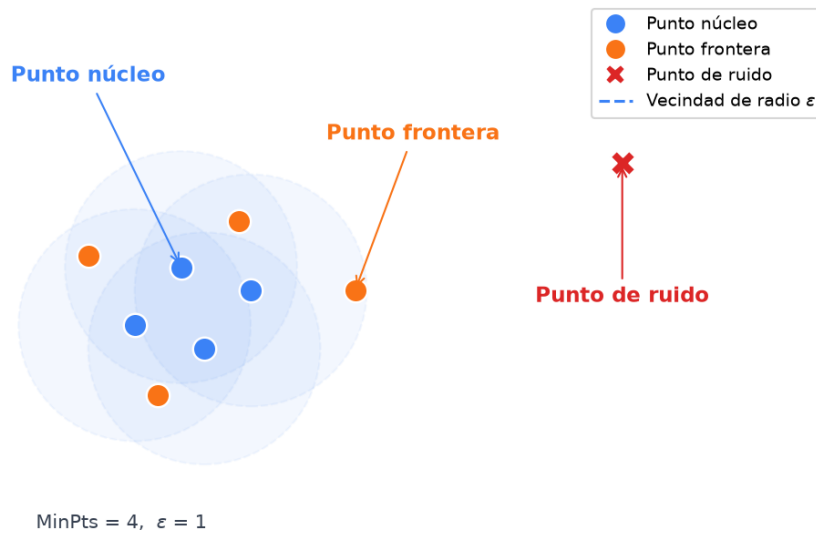


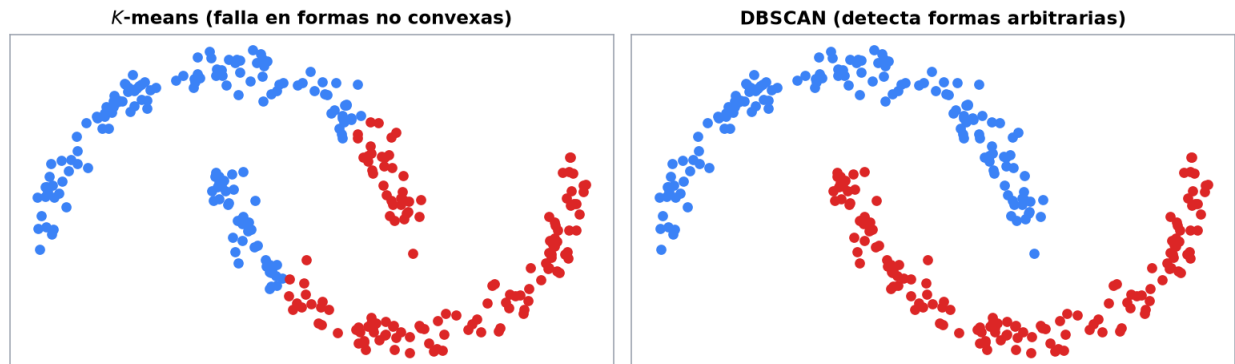
Figura8.6: Clasificación de puntos en DBSCAN: núcleo, frontera y ruido.

Ventajas sobre métodos basados en distancia

- **Robustez ante el ruido:** a diferencia de *K-means*, que obliga a cada muestra a pertenecer a un *cluster* (desplazando artificialmente los centroides ante valores atípicos), DBSCAN identifica y aísla el ruido de forma nativa.
- **Flexibilidad geométrica:** no asume que los *clusters* tienen que ser esféricos; puede descubrir *clusters* anidados, lineales o con formas complejas de baja dimensión dentro de espacios de alta dimensión.

La Fig. 8.7 compara ambos algoritmos sobre datos con forma no convexa: *K-means* corta el grupo por la mitad de forma arbitraria, mientras que DBSCAN respeta la forma real definida por la densidad.

K-means frente a DBSCAN en datos con forma de luna

Figura8.7: *K-means* frente a DBSCAN en datos con formas no convexas.

8.3 Técnicas de evaluación en clustering

La evaluación de las estructuras de agrupamiento es uno de los campos más complejos y debatidos del aprendizaje no supervisado, ya que, al no contar con etiquetas de clase o una “verdad fundamental” (*ground-truth*) durante el entrenamiento, no existe un criterio único y objetivo de éxito. La calidad de un agrupamiento a menudo se mide de forma subjetiva en función de la utilidad del modelo para el usuario final en su dominio de aplicación.

Sin embargo, para aproximar este análisis de manera rigurosa, la literatura científica ha consolidado métodos de evaluación divididos en cuatro grandes enfoques: (i) métricas geométricas internas, (ii) criterios basados en la teoría de la información, (iii) métricas y enfoques de información unificados y (iv) validaciones externas supervisadas.

8.3.1 Métricas de evaluación interna y estructura geométrica

Analizan la disposición espacial de los puntos en el espacio de características para medir dos propiedades deseables: que los *clusters* sean lo más compactos posibles (baja varianza interna) y que estén bien separados entre sí.

Coeficiente y *score* de silueta (*silhouette score*)

El coeficiente de silueta (*sc*) [Rou87] evalúa la calidad de la asignación de cada muestra de forma individual, sirviendo de diagnóstico para agrupamientos predominantemente esféricos. Para una instancia i , se calcula como:

$$sc(i) = \frac{b_i - a_i}{\max(a_i, b_i)}$$

Donde:

- a_i (**cohesión**): es la distancia promedio (usualmente euclídea) entre la instancia i y todos los demás puntos que pertenecen a su mismo cluster. Queremos que este valor sea lo más bajo posible.
- b_i (**separación**): es la distancia promedio entre la instancia i y todas las muestras del *cluster* vecino más cercano (el *cluster* que minimice este valor de distancia promedio, excluyendo el propio). Queremos que este valor sea lo más alto posible.

Interpretación de los resultados

- El coeficiente varía estrictamente en el rango de $[-1, 1]$.
- Un valor cercano a $+1$ indica que la muestra está en el interior de su propio grupo y muy alejada de las fronteras de otros clusters.
- Un valor cercano a 0 denota que la muestra se sitúa sobre la frontera de decisión entre dos clusters.
- Un valor cercano a -1 sugiere fuertemente que la muestra ha sido asignada al grupo incorrecto.

El *score* de silueta global es el promedio de los coeficientes de todas las instancias del conjunto de datos. En un diagrama de silueta, los coeficientes se ordenan de mayor a menor y se grafican por cluster. La altura de cada silueta (forma de cuchillo) indica el tamaño del *cluster* y el ancho representa el coeficiente individual de sus puntos. Esto permite identificar visualmente si algún *cluster* es excesivamente grande o si tiene demasiadas muestras por debajo del *score* promedio general (representado por una línea vertical discontinua), lo cual delata agrupamientos de baja calidad.

La Fig. 8.8 muestra un diagrama de silueta real con tres clusters: la mayoría de las muestras del cluster 1 (rojo) y 0 (azul) superan la silueta promedio, mientras que el cluster 2 (verde) es más heterogéneo.

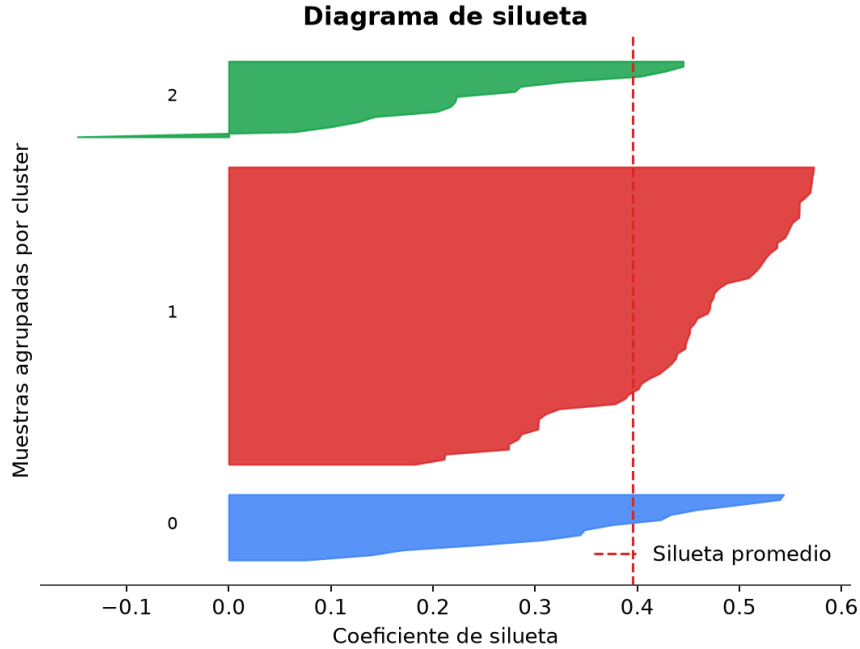


Figura8.8: Diagrama de silueta: cada “cuchillo” representa un cluster y su calidad de agrupamiento.

Estadístico gap (*gap statistic*)

Propuesto por Tibshirani et al. [TWH01], el estadístico *gap* formaliza matemáticamente la búsqueda del número óptimo de *clusters* (K) superando la vaguedad visual del método heurístico del codo (*elbow method*).

Compara la curva del logaritmo de la inercia observada de tus datos ($\log W_K$) con la esperanza matemática de la inercia calculada sobre múltiples muestras generadas artificialmente con una distribución uniforme (sin estructura de agrupamiento o hipótesis nula) en el hiperrectángulo que encierra a los datos reales:

$$\text{Gap}(K) = E^*[\log W_K] - \log W_K$$

El método estima el K óptimo maximizando este “*gap*” o brecha. Matemáticamente, la regla formal selecciona el menor K tal que:

$$K^* = \operatorname{argmin}_K \{K \mid \text{Gap}(K) \geq \text{Gap}(K+1) - s'_{K+1}\}$$

Donde $s'_{K+1} = s_{K+1} \sqrt{1 + 1/B}$ representa la desviación estándar de la simulación ajustada por el número de réplicas artificiales B . Su mayor ventaja es que puede determinar si el número óptimo de *clusters* es igual a 1 (es decir, que los datos no tienen *clusters* naturales), un escenario donde la inercia o el coeficiente de silueta fallan.

Coeficiente de correlación cofenética

Esta técnica evalúa la calidad de la estructura jerárquica impuesta por el *clustering* jerárquico. Mide la correlación de Pearson entre la matriz de disimilitudes iniciales $\{d_{ii'}\}$ de las observaciones y sus correspondientes disimilitudes cofenéticas $\{C_{ii'}\}$ obtenidas a partir de las alturas de fusión en el dendrograma.

La distancia cofenética $C_{ii'}$ es la altura exacta del enlace en el dendrograma donde las observaciones i e i' se unen por primera vez en un *cluster* común. Dado que las distancias cofenéticas deben cumplir estrictamente la desigualdad ultramétrica ($C_{ii'} \leq \max\{C_{ik}, C_{i'k}\}$), es poco común que los datos reales de entrada se acoplen perfectamente a esta restricción métrica. Una baja correlación cofenética advierte al diseñador que el dendrograma está forzando una jerarquía artificial donde los datos reales no la presentan.

8.3.2 Criterios probabilísticos y de teoría de la información

Cuando se trabaja con modelos de agrupamiento basados en probabilidad (como los modelos de mezclas Gaussianas o GMM entrenados mediante el algoritmo EM), las distancias geométricas puras dejan de ser fiables debido a que los grupos pueden adoptar formas elípticas, orientaciones oblicuas o densidades muy variadas.

Criterio de información de Akaike (AIC) y Bayesiano (BIC)

Si evaluamos un modelo probabilístico únicamente por su log-verosimilitud (LL), el modelo con mayor número de parámetros o componentes siempre parecerá mejor en los datos de entrenamiento, promoviendo el sobreajuste. Para evitar esto, AIC y BIC añaden penalizaciones matemáticas que balancean el ajuste y la complejidad:

$$AIC = -2LL + 2d$$

$$BIC = -2LL + d \log(N)$$

Donde LL es la log-verosimilitud máxima del modelo²⁵, d es el número total de parámetros libres a aprender (medias, pesos y covarianzas de los componentes) y N es el tamaño muestral de instancias.

Penalización:

Dado que $\log N > 2$ para cualquier conjunto de datos real ($N > 7$), el BIC penaliza la complejidad de forma mucho más severa que el AIC, priorizando modelos más parsimoniosos (con menor número de componentes).

Comportamiento asintótico:

El BIC es asintóticamente consistente, lo que significa que si el modelo real está dentro de los candidatos, la probabilidad de seleccionarlo tiende a 1 cuando $N \rightarrow \infty$. El AIC, por el contrario, tiende a sobreajustar en muestras infinitas, eligiendo modelos demasiado complejos, aunque en muestras pequeñas se comporta de forma más competitiva que el conservador BIC.

WAIC (*Widely Applicable Information Criterion*)

AIC y BIC asumen que los parámetros del modelo se aproximan a una distribución Gaussiana y que el modelo no es singular. Sin embargo, en mezclas probabilísticas y modelos de aprendizaje profundo con parámetros correlacionados o redundantes (modelos singulares), estas suposiciones fallan.

El WAIC es una métrica de información bayesiana que estima la densidad predictiva posterior esperada (ELPD) mediante métodos de simulación Monte Carlo, penalizando la complejidad a partir de la varianza posterior de las predicciones de cada punto de datos. Esto proporciona un marco de evaluación robusto que funciona incluso bajo geometrías de parámetros singulares y sobre-parametrizadas.

8.3.3 Métricas y enfoques de información unificados

El principio de mínima longitud de descripción (MDL)

Inspirado en la teoría de la información y la codificación de datos, el principio MDL evalúa el *clustering* bajo la premisa de la compresión de datos. Si un agrupamiento es natural y de alta calidad, debería permitirnos describir o transmitir el conjunto de datos de una forma más compacta (utilizando menos bits de información) que si describiéramos los puntos sin agrupar.

Para transmitir un conjunto de puntos usando MDL, debemos codificar dos elementos:

- La teoría o modelo: la ubicación espacial de los centroides de los *clusters* y sus respectivas asignaciones en bits ($\log_2 K$ bits por muestra).
- Los datos dado el modelo: los valores de las características de cada instancia, expresados únicamente como la desviación residual con respecto a su centroide asignado.

Si la segmentación captura grupos muy densos y reales, la drástica reducción de bits al describir los pequeños residuos individuales compensará con creces el costo extra de transmitir la posición de los centroides, logrando una descripción total minimalista. Si el agrupamiento no es real, el costo total en bits aumentará, señalando que la subdivisión es inútil.

Utilidad de categoría (*category utility*)

Utilizada en métodos de agrupamiento incremental y jerárquico como Cobweb o Classit, la utilidad de categoría (CU) mide el incremento en la capacidad de predecir correctamente los atributos de una muestra tras conocer que pertenece a un *cluster* específico, en contraste con la predicción incondicional de los datos³.

Para atributos categóricos o nominales, se formula como:

$$CU(C_1, \dots, C_k) = \frac{\sum_{l=1}^k \Pr[C_l] \sum_i \sum_j (\Pr[a_i = v_{ij} | C_l]^2 - \Pr[a_i = v_{ij}]^2)}{k}$$

Donde $\Pr[a_i = v_{ij} | C_l]$ es la probabilidad condicional de que el atributo i tome el valor v_{ij} en el *cluster* l , y $\Pr[a_i = v_{ij}]$ es su probabilidad incondicional global.

Evitar la fragmentación (división por k)

Sin el denominador k , la máxima puntuación se obtendría asignando cada instancia individual a su propio *cluster* exclusivo, donde la probabilidad condicional es 1.0. La división por k actúa como un factor heurístico para castigar la fragmentación excesiva del modelo.

Varianza mínima (*acuity*)

Para atributos continuos, la fórmula asume una distribución normal y requiere estimar desviaciones estándar locales y globales. Para evitar que un *cluster* con un único elemento (o con varianza cero) genere un valor infinito e indeterminado en la ecuación, se impone un parámetro de varianza mínima o *acuity*, que emula el error de medición del sensor.

8.3.4 Evaluación externa mediante datos etiquetados

Cuando disponemos de etiquetas de referencia externas (*labels* que no se utilizaron para guiar el *clustering*), podemos medir qué tan bien se acoplan las fronteras descubiertas con las clases biológicas o de negocio conocidas.

Pureza (*purity*)

Consiste en asignar a cada *cluster* completo la etiqueta de clase más frecuente en su interior y calcular la proporción de elementos clasificados correctamente. Se define formalmente como:

$$\text{Purity} = \sum_i \frac{N_i}{N} \max_j (p_{ij})$$

Donde N_i es el tamaño del *cluster* i , N es el total de datos, y $p_{ij} = N_{ij}/N_i$ es la fracción del *cluster* i ocupada por la clase j .

Limitación

La pureza no penaliza el número de clusters. Si asignas cada muestra de forma individual a su propio *cluster* ($K = N$), la pureza será de manera trivial igual a 1.0, invalidando su uso si no se controla de forma independiente el número de clusters.

Índice Rand (*Rand Index*, RI) y Rand ajustado (ARI)

El índice Rand analiza la consistencia del agrupamiento en función de parejas de elementos. Examina cada uno de los posibles pares de observaciones del *dataset* y evalúa si la decisión de asignarlos al mismo *cluster* o a *clusters* diferentes fue concordante en la partición estimada (U) y en la de referencia (V):

$$R = \frac{TP+TN}{TP+FP+FN+TN}$$

- **TP** (Verdaderos Positivos): número de pares que están en el mismo *cluster* en U y en la misma clase en V .
- **TN** (Verdaderos Negativos): número de pares que están en *clusters* separados en U y en diferentes clases en V .
- **FP** (Falsos Positivos): número de pares asignados al mismo *cluster* en U pero de diferentes clases en V .
- **FN** (Falsos Negativos): número de pares en *clusters* separados en U pero de la misma clase en V .

El RI oscila entre 0 (desacuerdo total) y 1 (concordancia perfecta). Sin embargo, el RI clásico raramente alcanza el valor 0, ya que el azar tiende a inflar su puntuación. El Índice Rand Ajustado (ARI) corrige esto restando la concordancia esperada de dos clasificaciones aleatorias bajo un modelo de distribución hipergeométrica generalizada, asegurando que un agrupamiento al azar obtenga una puntuación promedio de 0.0.

Clases a clusters (*Classes-to-Clusters Evaluation*)

Popularizado por suites de software clásico como Weka, este enfoque entrena el modelo de *clustering* ignorando por completo el atributo de clase. Una vez que el espacio se ha segmentado, el evaluador examina la composición de cada *cluster* para asignarle de forma retrospectiva la etiqueta de la clase mayoritaria.

A partir de esta asignación, es posible mapear las fronteras geométricas como si fueran un clasificador supervisado y construir una matriz de confusión clásica que revele las tasas exactas de falsos positivos, falsos negativos y precisión global por clase.

8.4 Ejemplo práctico en Java con SMILE

El repositorio `programacion-avanzada-smile` incluye el ejemplo completo `Ejemplo05Clustering.java`, que aplica los tres algoritmos de esta sección con la librería **SMILE**. Se usa el conjunto *Iris* **quitando la columna de la especie**: el algoritmo no ve las etiquetas, pero como sabemos que hay tres especies podemos comparar la partición obtenida con la real mediante un índice externo (ARI).

```
import java.util.Arrays;
import smile.clustering.CentroidClustering;
import smile.clustering.Clustering;
import smile.clustering.DBSCAN;
import smile.clustering.HierarchicalClustering;
import smile.clustering.KMeans;
import smile.clustering.linkage.WardLinkage;
import smile.data.DataFrame;
import smile.io.Read;
import smile.math.MathEx;
import smile.validation.metric.AdjustedRandIndex;

MathEx.setSeed(42);

DataFrame iris = Read.csv("data/iris.csv", "header=true").factorize("species");
double[][] x = iris.drop("species").toArray(); // sin etiquetas
int[] especieReal = iris.column("species").toIntArray(); // solo para evaluar

// 1. K-means con k = 3: fit(datos, k, maxIteraciones)
CentroidClustering<double[], double[]> kmeans = KMeans.fit(x, 3, 100);
int[] grupos = kmeans.group();
System.out.printf("Distorsion (inerencia) = %.3f%n", kmeans.distortion());
System.out.printf("ARI frente a las especies reales = %.4f%n",
    AdjustedRandIndex.of(especieReal, grupos));

// Asignar una muestra nueva al centroide mas cercano
double[] nueva = { 5.9, 3.0, 5.1, 1.8 };
System.out.printf("La muestra cae en el cluster %d%n", kmeans.predict(nueva));

// 2. Metodo del codo: distorsion para distintos valores de k
for (int k = 2; k <= 8; k++) {
    System.out.printf("k = %d  distorsion = %.3f%n", k, KMeans.fit(x, k, 100).
        ↪distortion());
}

// 3. DBSCAN: fit(datos, minPts, epsilon). No hay que fijar k y detecta ruido
DBSCAN<double[]> dbscan = DBSCAN.fit(x, 5, 0.8);
long ruido = Arrays.stream(dbscan.group()).filter(g -> g == Clustering.OUTLIER).
    ↪count();
System.out.printf("DBSCAN: %d clusters, %d puntos de ruido%n", dbscan.k(), ruido);

// 4. Clustering jerarquico aglomerativo (enlace de Ward) y corte en 3 grupos
HierarchicalClustering jerarquico = HierarchicalClustering.fit(WardLinkage.of(x));
int[] particion3 = jerarquico.partition(3);
System.out.printf("Jerarquico: ARI = %.4f%n", AdjustedRandIndex.of(especieReal, ↪
    ↪particion3));
```

Algunas ideas que conviene observar al ejecutarlo:

- `kmeans.distortion()` es la inercia intracluster (WCSS) que minimiza *K-means*; el bucle sobre *k* genera los valores que se representarían en la gráfica del método del codo.

- Los puntos que DBSCAN considera ruido reciben la etiqueta especial `Clustering.OUTLIER` en lugar de un número de *cluster*.
- En el *clustering* jerárquico, primero se construye el criterio de enlace (`WardLinkage`, aunque también existen `SingleLinkage`, `CompleteLinkage` o `UPGMALinkage` para el enlace promedio) y después se corta el dendrograma con `partition(k)`.
- El ejemplo completo compara también *X-means* (elección automática de k mediante BIC) y muestra el efecto de estandarizar las variables antes de agrupar.

Para ejecutarlo desde la raíz del repositorio: `mvn exec:java -Dexec.mainClass=es.usal.smile.Ejemplo05Clustering`.

8.5 Resumen

- *K-means*: simple, rápido, requiere K conocido.
- **Método del codo**: heurística para elegir K .
- *Clustering jerárquico*: dendrograma, mejor visualización.
- **DBSCAN**: detecta formas arbitrarias y outliers.
- **Silhueta**: métrica para evaluar calidad sin etiquetas.

Siguiente: En 3.2 aprenderemos a reducir dimensionalidad para simplificar y visualizar datos complejos.

3.2 REDUCCIÓN DE DIMENSIONALIDAD

9.1 Introducción

En la analítica de datos moderna y el diseño de sistemas de inteligencia artificial, es común enfrentarse a conjuntos de datos caracterizados por cientos o miles de variables predictoras. Sin embargo, trabajar de forma directa en estos espacios multidimensionales presenta graves inconvenientes prácticos y teóricos. La reducción de dimensionalidad aborda de manera sistemática este problema buscando proyectar o comprimir los datos en un espacio de baja dimensión (habitualmente 2D o 3D). Este proceso no solo acelera drásticamente la computación y mitiga el sobreajuste (*overfitting*), sino que es una herramienta indispensable para la visualización de datos (*Data Viz*) y la extracción de factores latentes explicables.

9.2 La maldición de la dimensionalidad (*the curse of dimensionality*)

La intuición humana está programada para razonar en tres dimensiones físicas, lo que nos dificulta comprender las propiedades geométricas de los espacios de alta dimensionalidad. Muchas propiedades matemáticas de los algoritmos tradicionales de aprendizaje automático se degradan o colapsan debido a este fenómeno, acuñado formalmente por Richard Bellman como la maldición de la dimensionalidad.

La Fig. 9.1 muestra la progresión desde un punto hasta un hipercubo de p dimensiones: a medida que crece la dimensión, el espacio se vuelve ultra-disperso y las muestras tienden a situarse en la frontera.

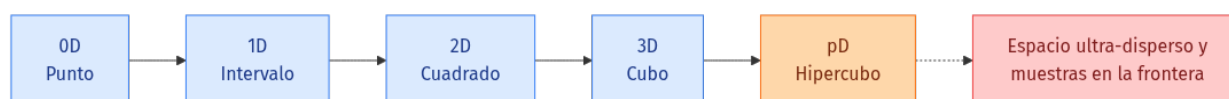


Figura9.1: Progresión de dimensiones: del punto al hipercubo.

9.2.1 Pérdida de vecindad y espacio disperso

El principal síntoma de la maldición de la dimensionalidad es que, a medida que aumenta la dimensión p , el volumen del espacio crece de forma exponencial con respecto a las características. Esto provoca que cualquier conjunto de datos real, por grande que sea, popule de manera extremadamente dispersa el espacio de entrada. Para ilustrar este hecho:

- Si en una sola dimensión ($p = 1$) un rango local del 10% representa una muestra representativa de vecindad, para capturar el mismo volumen equivalente del 10% en un espacio de 10 dimensiones ($p = 10$), un algoritmo local (como K -NN) debe extender su búsqueda cubriendo el 80% del rango de cada una de las variables.
- De este modo, los puntos del vecindario dejan de ser “locales” y el algoritmo pierde su poder estadístico estimador.
- Para mantener una densidad de muestreo constante al añadir variables predictoras, el tamaño del *dataset* requerido crece exponencialmente ($O(N^p)$). En la práctica, obtener un volumen tal de datos es inviable.

9.2.2 La anomalía geométrica de los extremos

En espacios de alta dimensionalidad, la geometría se vuelve muy contraintuitiva.

- **Atracción por la frontera:** en un cuadrado unitario de dos dimensiones (1x1), la probabilidad de que una muestra elegida al azar se sitúe cerca del borde (a menos de 0,001 de distancia) es de apenas un 0,4%. Sin embargo, en un hipercubo de 10000 dimensiones, esta probabilidad es superior al 99,9999%. Prácticamente todas las muestras residen de forma natural en los bordes y esquinas exteriores del hipercubo.
- **Uniformidad de distancias:** la distancia promedio entre dos puntos elegidos al azar en un hipercubo crece drásticamente con la dimensionalidad. Las distancias relativas se igualan, haciendo que todas las muestras parezcan casi a la misma distancia de las demás y neutralizando la eficacia de las métricas estándar como la distancia euclídea. Esto incrementa severamente el riesgo de sobreajuste de los clasificadores, ya que el modelo se vuelve inestable ante ligeras variaciones.

La Fig. 9.2 cuantifica ambos efectos con simulaciones: a la izquierda, el porcentaje de muestras cerca del borde de un hipercubo crece rápidamente con las dimensiones; a la derecha, el ratio entre la distancia mínima y máxima entre puntos aleatorios tiende a 1, es decir, todos los puntos parecen equidistantes.

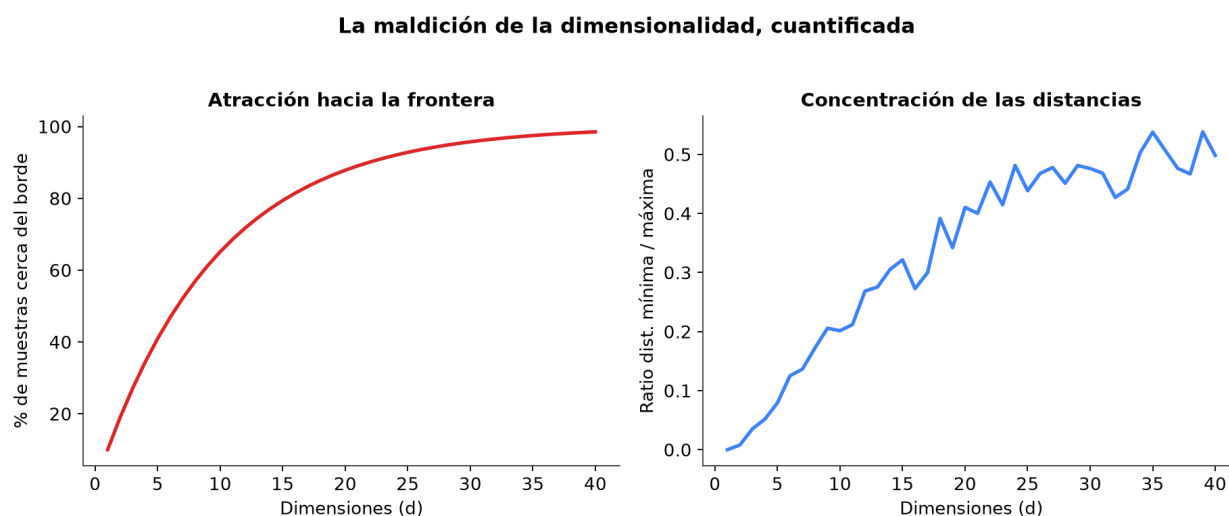


Figura9.2: La maldición de la dimensionalidad, cuantificada mediante simulación.

9.3 Métodos de proyección lineal: PCA (*Principal Components Analysis*)

El análisis de componentes principales (PCA), desarrollado originalmente a principios del siglo XX por Pearson y Hotelling [Hot33, Pea01], es el algoritmo de reducción de dimensionalidad lineal por excelencia. Su objetivo es proyectar ortogonalmente los datos originales $X \in \mathbb{R}^D$ en un subespacio lineal de baja dimensión $Z \in \mathbb{R}^M$ (donde $M < D$), minimizando la pérdida de información bajo criterios estadísticos estrictos.

9.3.1 Perspectiva de máxima varianza

Desde el punto de vista geométrico, la primera componente principal (PC1) se define como la dirección o eje del espacio de características a lo largo de la cual los datos varían más. Al proyectar los datos sobre este eje, se preserva el mayor porcentaje de la dispersión de la información original.

La Fig. 9.3 compara dos proyecciones de los mismos datos: sobre PC1 (izquierda) los puntos proyectados quedan muy dispersos a lo largo del eje y se conserva el 90% de la varianza; sobre otra dirección (derecha) los puntos se amontonan y solo se retiene el 13%.

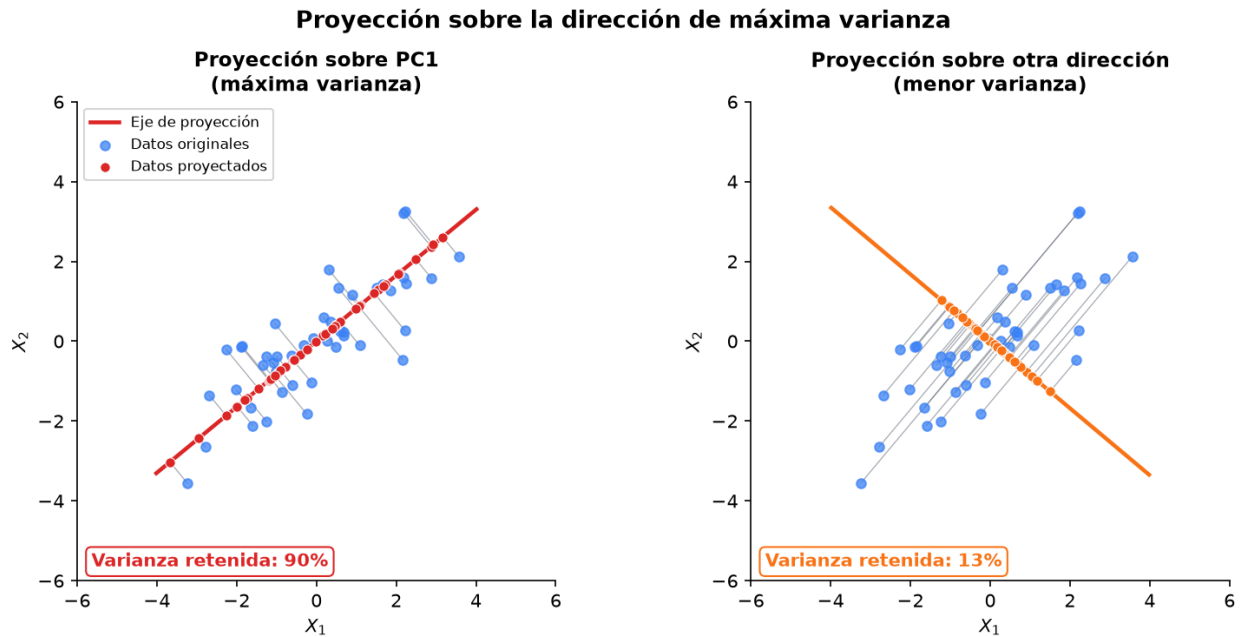


Figura 9.3: Proyección sobre la dirección de máxima varianza (PC1) frente a otra dirección.

La segunda componente principal (PC2) busca la dirección que explique la mayor cantidad posible de la varianza restante bajo la restricción estricta de ser completamente ortogonal (y, por tanto, incorrelacionada) a la primera componente.

Este procedimiento se repite de manera sucesiva para generar hasta D componentes distintas.

La Fig. 9.4 muestra este concepto sobre datos reales correlacionados: la flecha roja (PC1) señala la dirección de máxima varianza y la verde (PC2), ortogonal a la anterior, captura la varianza restante.

9.3.2 Derivación matemática y la SVD

Para realizar PCA, es imperativo que los datos originales sean previamente centrados (restando la media aritmética de cada variable) y, usualmente, estandarizados para que tengan una varianza unitaria (evitando que variables con escalas métricas arbitrariamente grandes dominen la optimización).

La matriz de covarianza de los datos se define como:

$$S = \frac{1}{N} X X^T$$

Mediante la Descomposición en valores propios de la matriz de covarianza (o a través de la Descomposición en Valores Singulares - SVD de la matriz de datos original X), se extraen las direcciones principales:

$$S = V D^2 V^T$$

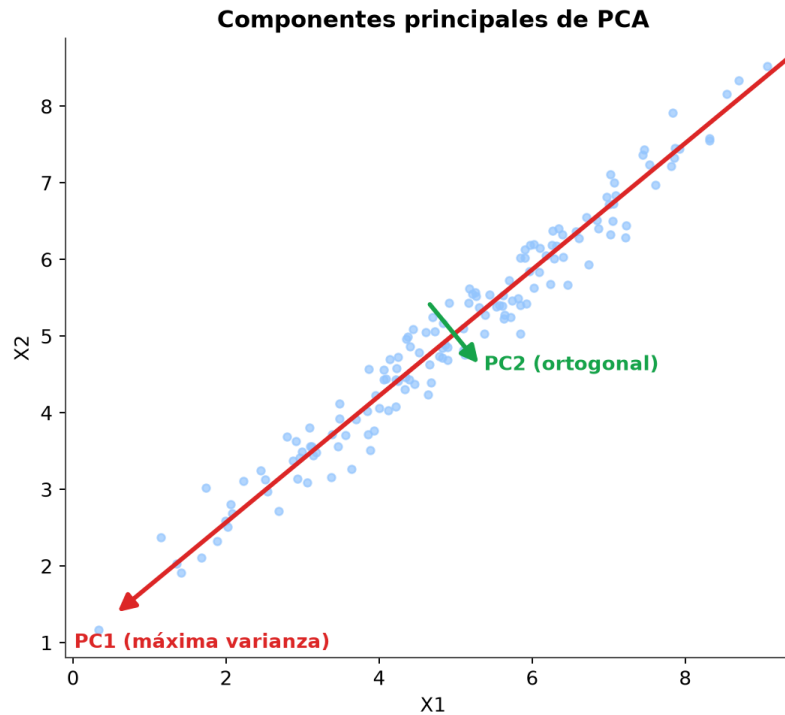


Figura 9.4: Componentes principales: PC1 captura la máxima varianza, PC2 es ortogonal.

Donde las columnas de la matriz V corresponden a los vectores de carga (*loadings*), que representan las direcciones de las componentes principales. El valor propio λ_m correspondiente a cada autovector mide directamente la cantidad de varianza explicada por ese componente específico.

9.3.3 Compresión, reconstrucción y perspectiva de autoensamblador lineal

PCA puede conceptualizarse matemáticamente como un autoencoder lineal. Consta de dos fases principales:

- **Codificador** (*encoder*): convierte el vector de entrada x_n en una representación reducida $z_n = B^T x_n$, donde B contiene los autovectores con mayores autovalores asociados.
- **Decodificador** (*decoder*): reconstruye la proyección aproximada de vuelta al espacio original: $\hat{x}_n = B z_n$.

El error promedio de reconstrucción o distorsión cuadrática minimizado por este procedimiento equivale exactamente a la suma de los valores propios de las componentes que han sido descartadas del análisis.

Un ejemplo real de la utilidad de PCA aparece en el conjunto de datos *NCI60*, donde cada muestra tiene 6830 genes (dimensiones). La Fig. 9.5 proyecta estas muestras sobre sus tres primeras componentes principales: pese a la enorme dimensionalidad original, las líneas celulares del mismo tipo de cáncer (mismo color) tienden a agruparse en este espacio reducido.

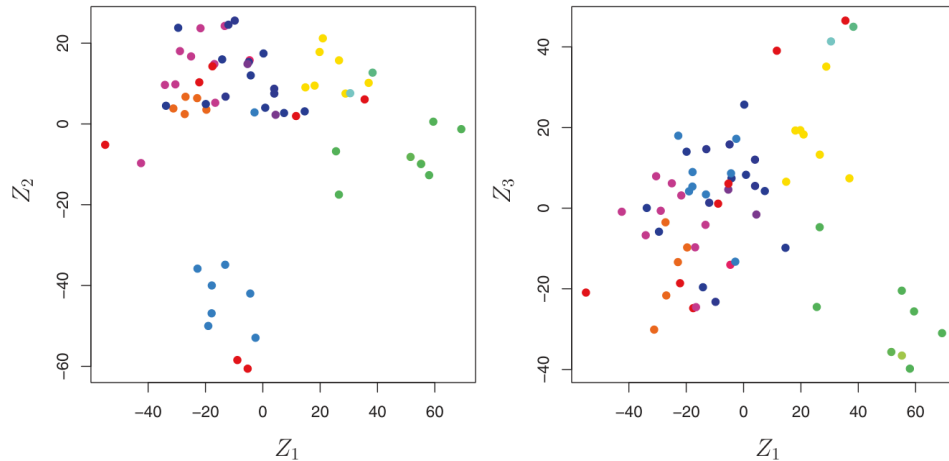


Figura 9.5: Proyección de las líneas celulares de cáncer *NCI60* (6830 genes) sobre sus tres primeras componentes principales.
Fuente: James, G., Witten, D., Hastie, T., & Tibshirani, R. (2013). *An Introduction to Statistical Learning*, Figura 10.15. Springer.
Libro de libre distribución para uso educativo (statlearning.com).

9.4 Métodos no lineales y aprendizaje de variedades (*manifold learning*)

Aunque PCA es sumamente robusto y rápido, sufre una limitación obvia: asume que el subespacio interesante de los datos es lineal (un hiperplano plano). Cuando las relaciones de los datos son intrínsecamente no lineales, como en el clásico ejemplo del *Swiss roll* (un rollo de papel curvo en 3D), la proyección ortogonal lineal de PCA colapsa los puntos distantes y mezcla artificialmente la información.

9.4.1 La hipótesis de la variedad (*the manifold hypothesis*)

El aprendizaje de variedades (*manifold learning*) asume la validez de la hipótesis de la variedad: todos los datos naturales del mundo real (imágenes de MNIST, rostros, voces o textos) yacen en una variedad de baja dimensión embebida dentro del espacio original de alta dimensión.

- Una variedad (*manifold*) es una superficie continua y curva que, de manera local, se asemeja a un espacio lineal euclidiano plano.
- Por ejemplo, el espacio de imágenes de dígitos escritos a mano (28×28 píxeles = 784 dimensiones) está restringido por leyes físicas e invarianzas (grosor del trazo, inclinación, continuidad) que reducen enormemente los grados de libertad reales del sistema, confinando las muestras viables a una variedad continua de muy baja dimensión.
- El objetivo del *manifold learning* no paramétrico es aprender coordenadas incrustadas para cada punto, de modo que se represente fielmente la topología interna y la distancia a lo largo de la superficie de la variedad.

La Fig. 9.6 muestra el ejemplo clásico del *Swiss roll*: los datos (izquierda) están enrollados en 3D, pero su estructura real es una superficie 2D que puede “desenrollarse” (derecha) preservando las distancias a lo largo de la variedad.

Swiss roll: motivación del manifold learning

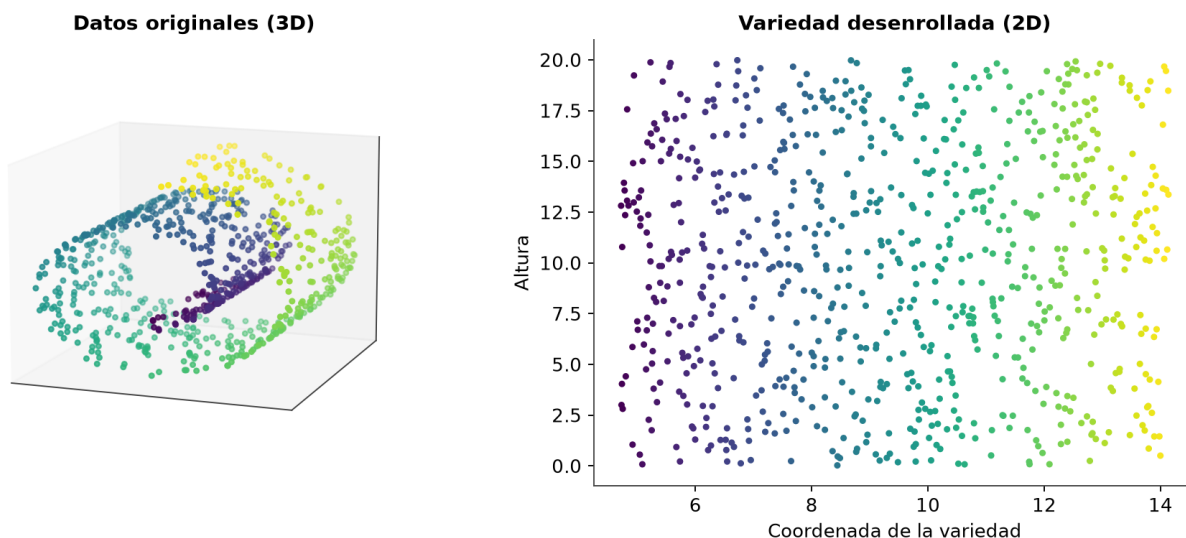


Figura 9.6: El *Swiss roll*: datos enrollados en 3D (izquierda) y su variedad desenrollada en 2D (derecha).

9.4.2 Algoritmos no lineales clave

Kernel PCA (kPCA)

Para extender PCA a problemas no lineales, se utiliza el truco del kernel. kPCA mapea de forma implícita los datos de entrada a un espacio de Hilbert de dimensiones ultra-altas (o infinitas) $\Phi(x)$, donde las relaciones no lineales complejas se vuelven linealmente separables o proyectables.

- Utilizando funciones de kernel comunes como el RBF (Gaussiano), se pueden modelar proyecciones complejas.
- **Inconveniente:** a diferencia del PCA estándar, kPCA no define un mapeo de proyección directa e invertible para datos fuera de la muestra (*out-of-sample prediction*), y en ocasiones, *kernels* mal ajustados pueden expandir y distorsionar el espacio en lugar de comprimirlo de forma útil.

La Fig. 9.7 ilustra el truco del *kernel* con un ejemplo clásico: dos clases dispuestas en círculos concéntricos (izquierda) son imposibles de separar con una línea recta en 2D. Al aplicar el mapeo $\phi(x_1, x_2) = (x_1, x_2, x_1^2 + x_2^2)$ (derecha), las clases quedan a alturas distintas y un simple plano horizontal las separa perfectamente.

t-SNE (*t-distributed Stochastic Neighbor Embedding*)

Propuesto por Maaten y Hinton (2008) [vdMH08], t-SNE es la técnica no convexa preferida para la visualización cualitativa de agrupamientos complejos en dos dimensiones.

1. **Espacio de alta dimensión** (SNE original): convierte las distancias euclidianas entre muestras en probabilidades condicionales Gaussianas $p_{j|i}$ que denotan similitud. Los puntos cercanos reciben altas probabilidades de vecindad y los lejanos probabilidades infinitesimales.
2. **El problema del hacinamiento** (*crowding problem*): cuando se proyectan datos de alta dimensión a un espacio plano 2D, el volumen del espacio disponible disminuye de forma exponencial. Las distancias medias crecen tanto que, usando aproximaciones normales, las fuerzas de atracción obligan a todos los puntos distantes a agruparse en un núcleo denso e indistinguible en el centro del gráfico.

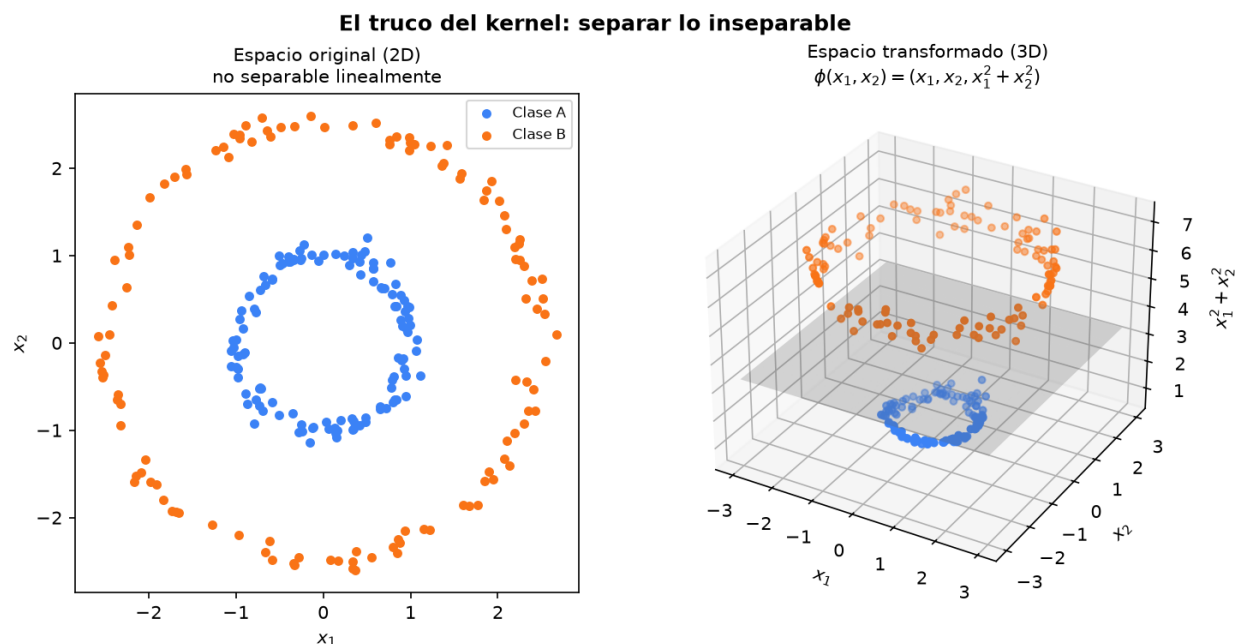


Figura9.7: El truco del *kernel*: datos no separables en 2D se vuelven separables al proyectarlos a una dimensión adicional.

3. **La solución de la distribución *t* de student:** *t*-SNE resuelve esta limitación utilizando una distribución *t* de student con un grado de libertad (equivalente a una distribución Cauchy) en el espacio de baja dimensión⁵¹. Al tener colas mucho más pesadas e invertidas en el denominador de la ecuación de probabilidad latente q_{ij} , se eliminan las fuerzas de atracción no deseadas entre *clusters* distantes:

$$q_{ij} = \frac{(1 + \|z_i - z_j\|^2)^{-1}}{\sum_{k \neq l} (1 + \|z_k - z_l\|^2)^{-1}}$$

El gradiente actúa de manera similar a una ley física de repulsión-atracción (similar a fuerzas de galaxias y estrellas), permitiendo que los *clusters* se organicen y separen de forma óptima en el plano visual.

La Fig. 9.8 compara ambos métodos sobre un ejemplo clásico: dos “lunas” entrelazadas embebidas en un espacio de 30 dimensiones. PCA (izquierda), al ser una proyección lineal, únicamente rota los datos y no logra separar las dos clases, que siguen entrelazadas. *t*-SNE (derecha) reorganiza los puntos de forma no lineal preservando las vecindades locales, consiguiendo dos grupos claramente diferenciados.

UMAP (Uniform Manifold Approximation and Projection)

UMAP [MHM18] es una de las técnicas de aprendizaje de variedades más potentes de la actualidad. Fundamentada en la geometría riemanniana clásica y la topología algebraica, UMAP asume que el espacio de los datos es localmente conexo y que la variedad sobre la que yacen es uniforme.

A diferencia de *t*-SNE, que se enfoca casi exclusivamente en retener vecindades muy locales (relaciones de corto alcance), UMAP es capaz de preservar tanto la estructura local como la estructura global de los datos.

Es matemáticamente mucho más eficiente, lo que se traduce en una velocidad de ejecución sustancialmente mayor sobre conjuntos de datos masivos con millones de muestras.

La Fig. 9.9 muestra una proyección UMAP a 2D del conjunto de datos de dígitos manuscritos (64 dimensiones): cada dígito (0-9, un color por clase) forma un grupo compacto y bien diferenciado, y la posición relativa entre grupos también resulta informativa.

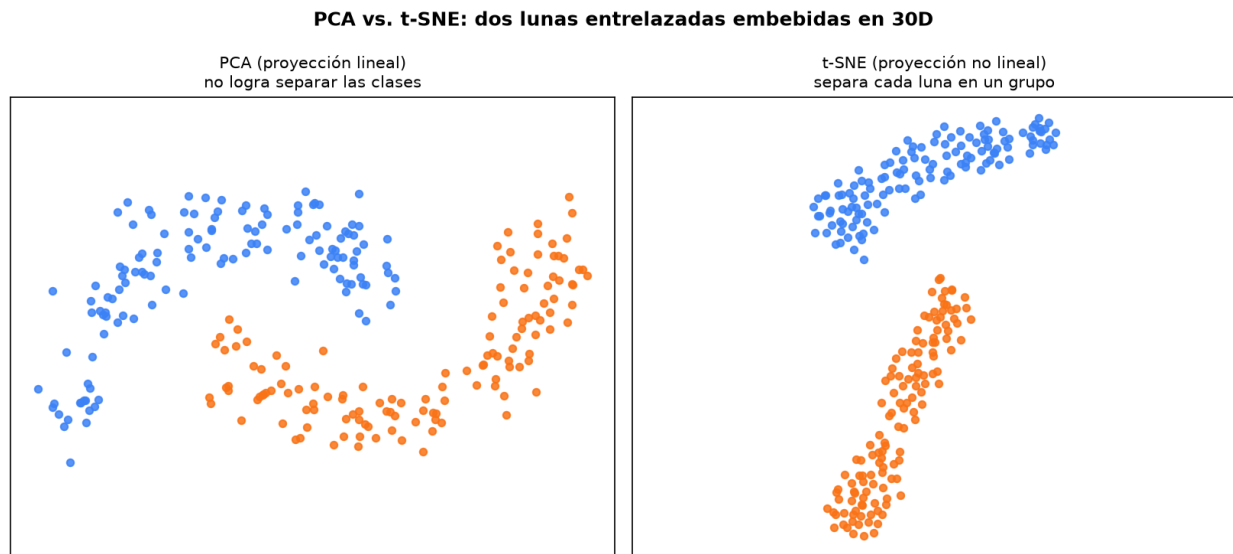


Figura9.8: PCA (proyección lineal) frente a t -SNE (proyección no lineal) sobre datos con estructura no lineal.

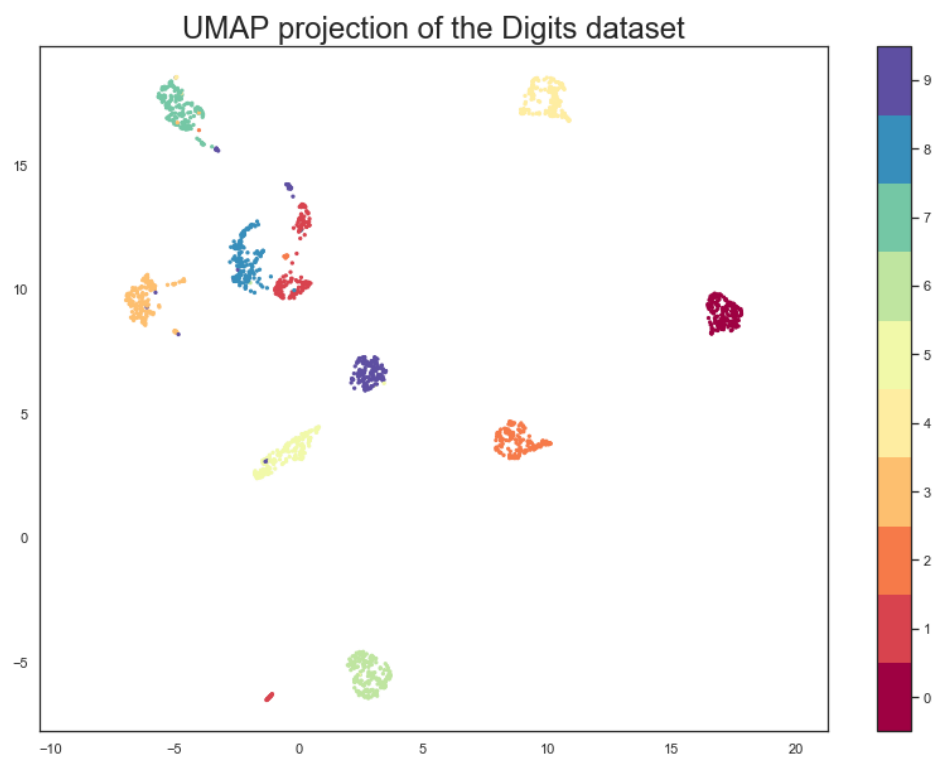


Figura9.9: Proyección UMAP del conjunto de datos *Digits*. Fuente: documentación de umap-learn, «How to Use UMAP» (umap-learn.readthedocs.io). Copyright (c) 2017, Leland McInnes. Licencia BSD de 3 cláusulas.

9.5 Ejemplo práctico en Java con SMILE

El repositorio `programacion-avanzada-smile` incluye el ejemplo completo `Ejemplo06PCA.java`, que aplica PCA al conjunto *Iris* (cuatro variables) con la librería **SMILE**. El siguiente fragmento resume sus pasos principales.

```
import java.util.Arrays;
import smile.classification.KNN;
import smile.data.DataFrame;
import smile.feature.extraction.PCA;
import smile.io.Read;
import smile.tensor.Vector;
import smile.validation.metric.Accuracy;

DataFrame iris = Read.csv("data/iris.csv", "header=true").factorize("species");
double[][] x = iris.drop("species").toArray();
int[] y = iris.column("species").toIntArray();

// 1. Ajuste del PCA sobre la matriz de covarianzas.
// PCA.cor(x) usaria la matriz de correlaciones (equivale a estandarizar antes).
PCA pca = PCA.fit(x);

// 2. Varianza explicada por cada componente (el "scree plot" en forma de tabla)
Vector proporcion = pca.varianceProportion();
Vector acumulada = pca.cumulativeVarianceProportion();
for (int i = 0; i < proporcion.size(); i++) {
    System.out.printf("PC%d: %.2f%% (acumulada %.2f%%)\n",
        i + 1, 100 * proporcion.get(i), 100 * acumulada.get(i));
}
System.out.println(pca.loadings()); // cargas: peso de cada variable en cada
    componente

// 3. Proyeccion de 4 dimensiones a 2
PCA proyeccion2D = pca.getProjection(2);
double[][] x2 = proyeccion2D.apply(x);
System.out.println("Primera muestra proyectada: " + Arrays.toString(x2[0]));

// Alternativa: pedir las componentes necesarias para retener el 95% de la varianza
int componentes95 = pca.getProjection(0.95).apply(x)[0].length;

// 4. PCA como paso previo a un clasificador.
// El PCA se ajusta SOLO con entrenamiento y se aplica despues al test:
// ajustarlo con todos los datos seria una fuga de informacion.
Utiles.ParticionArrays particion = Utiles.split(x, y, 0.7);
PCA pcaTrain = PCA.fit(particion.xTrain()).getProjection(2);
double[][] xTrain2 = pcaTrain.apply(particion.xTrain());
double[][] xTest2 = pcaTrain.apply(particion.xTest());

KNN<double[]> knnCompleto = KNN.fit(particion.xTrain(), particion.yTrain(), 5);
KNN<double[]> knnReducido = KNN.fit(xTrain2, particion.yTrain(), 5);
System.out.printf("KNN con 4 variables    accuracy = %.2f%%\n",
    100.0 * Accuracy.of(particion.yTest(), knnCompleto.predict(particion.
        xTest())));
System.out.printf("KNN con 2 componentes  accuracy = %.2f%%\n",
    100.0 * Accuracy.of(particion.yTest(), knnReducido.predict(xTest2)));
```

Algunas ideas que conviene observar al ejecutarlo:

- `varianceProportion()` y `cumulativeVarianceProportion()` son el equivalente numérico del

gráfico de sedimentación (*scree plot*) y permiten decidir cuántas componentes conservar.

- `getProjection(2)` fija el número de componentes, mientras que `getProjection(0.95)` lo elige automáticamente a partir de la varianza que se quiere retener.
- Al usar PCA como preprocesado, ajustarlo con todos los datos (entrenamiento y test) sería una **fuga de información**: el PCA debe ajustarse solo con el conjunto de entrenamiento.
- El ejemplo completo incluye también *kernel PCA* (`KernelPCA` con un núcleo gaussiano) y PCA probabilístico; SMILE ofrece además *t*-SNE y UMAP en el paquete `smile.manifold`.

Para ejecutarlo desde la raíz del repositorio: `mvn exec:java -Dexec.mainClass=es.usal.smile.Ejemplo06PCA`.

9.6 Resumen

- **PCA**: rápido, lineal, interpretable características originales.
- ***t*-SNE**: visualización excelente, no lineal, lento para datos grandes.
- **UMAP**: balance entre PCA y *t*-SNE.
- **Varianza explicada**: métrica para elegir número de componentes.
- **Maldición de la dimensionalidad**: motiva reducción.
- **Aplicaciones**: compresión, visualización, preprocesamiento.

Siguiente: En 3.3 aprenderemos a detectar anomalías, el último tema en aprendizaje no supervisado.

3.3 DETECCIÓN DE ANOMALÍAS

10.1 Introducción

La detección de anomalías es una tarea no supervisada de gran relevancia industrial orientada a identificar patrones inusuales o eventos sospechosos dentro de un flujo continuo de datos. Se aplica ampliamente en la detección de fraudes financieros, la prevención de intrusiones en redes corporativas, el control de calidad en manufactura y la limpieza automatizada de conjuntos de datos.

10.2 Filosofía de la detección de anomalías

A nivel conceptual, la detección de anomalías se basa en la premisa de que los comportamientos normales (denominados *inliers*) son altamente frecuentes y comparten patrones de comportamiento consistentes, mientras que las anomalías (*outliers*) son eventos raros cuyas características se desvían de manera significativa de la norma.

10.2.1 *Anomaly detection vs. novelty detection*

Aunque en la práctica estos términos se utilicen en ocasiones como sinónimos, la literatura especializada establece una distinción metodológica crítica basada en la contaminación del conjunto de datos de entrenamiento:

- **Detección de anomalías** (*anomaly/outlier detection*): el algoritmo se entrena sobre un conjunto de datos que no está depurado y que, por tanto, puede contener un porcentaje desconocido de *outliers* o muestras ruidosas infiltradas de forma natural. El objetivo es tanto identificar estos *outliers* dentro del propio *set* de entrenamiento (limpieza de datos) como clasificar correctamente las nuevas observaciones entrantes.
- **Detección de novedades** (*novelty detection*): el algoritmo se entrena bajo la premisa estricta de que el *dataset* de entrenamiento está completamente limpio y exento de anomalías. El objetivo aquí es definir de forma precisa la frontera del comportamiento “normal” conocido para catalogar cualquier patrón nuevo o desconocido como una “novedad”.

10.2.2 El enfoque no supervisado

La detección de anomalías se aborda fundamentalmente mediante aprendizaje no supervisado debido a que, en situaciones del mundo real, no se conoce de antemano qué tipo de anomalía puede ocurrir. Intentar entrenar un clasificador supervisado tradicional (como una regresión logística) resulta ineficaz debido al severo desbalance de clases y a que las anomalías futuras pueden presentar patrones completamente inéditos que el modelo jamás vio en el entrenamiento.

10.3 Enfoques estadísticos y de reconstrucción

Antes de recurrir a modelos más complejos, existen dos aproximaciones clásicas muy eficientes basadas en la estimación de densidad y en la proyección lineal de los datos.

[Datos Normales (Inliers)] —► Modelado de Densidad/Proyección —► Umbral Establecido | [Anomalía (Baja Densidad/Alto Error)] —► Alerta / Outlier

10.3.1 Modelado de densidad mediante mezclas gaussianas (GMM)

Bajo este enfoque, se asume que las observaciones normales se concentran en regiones del espacio de características que presentan una alta densidad de probabilidad.

- Se entrena un **Modelo de Mezcla Gaussiana** (GMM) para aproximar la función de densidad de probabilidad del *dataset*.
- Para clasificar una nueva muestra, se calcula su **densidad** bajo el modelo estimado. Cualquier instancia ubicada en una región de baja densidad por debajo de un umbral preestablecido se marca como anomalía.
- En entornos reales donde la tasa histórica de fallos es conocida (por ejemplo, un 4% de productos defectuosos en una fábrica), el **umbral** se establece de forma matemática seleccionando el percentil correspondiente (el 4% con menor densidad bajo el modelo).

La Fig. 10.1 ilustra el concepto: los contornos azules representan curvas de igual densidad de probabilidad y los puntos rojos marcados con X son anomalías que caen fuera de las regiones de alta densidad.

10.3.2 Enfoque de reconstrucción utilizando PCA

Esta técnica se basa en el principio de que los componentes principales mayoritarios de un análisis PCA capturan las direcciones de máxima varianza que caracterizan al comportamiento normal del sistema.

- El *dataset* se proyecta a un espacio de baja dimensión utilizando PCA y, posteriormente, se reconstruye de vuelta al espacio original utilizando la matriz inversa.
- Para cada instancia, se calcula el **error de reconstrucción** (la distancia cuadrática entre el vector original x y su reconstrucción $\hat{x}$).
- Dado que las componentes principales no capturan las desviaciones inusuales de los *outliers*, las anomalías experimentarán un error de reconstrucción significativamente mayor que las instancias normales, permitiendo su fácil identificación.

La Fig. 10.2 ilustra el proceso: los datos normales (azul) se proyectan sobre la línea de PC1 y se reconstruyen con un error mínimo (líneas finas), mientras que las dos anomalías (rojo) están lejos de la dirección principal de los datos y, al reconstruirse sobre esa misma línea, presentan un error de reconstrucción mucho mayor (flechas largas).

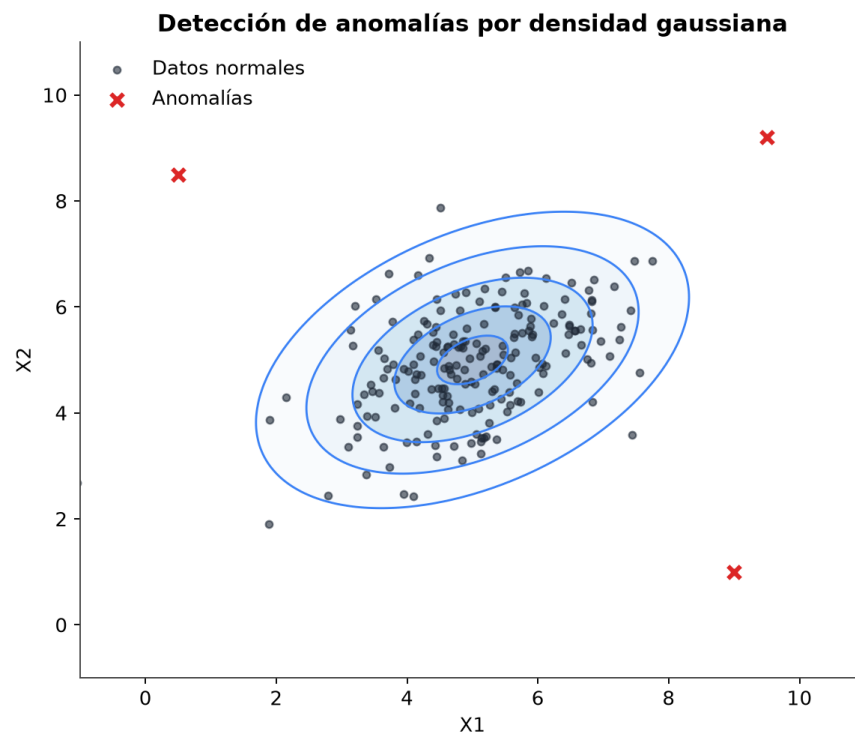


Figura10.1: Detección de anomalías mediante estimación de densidad gaussiana.

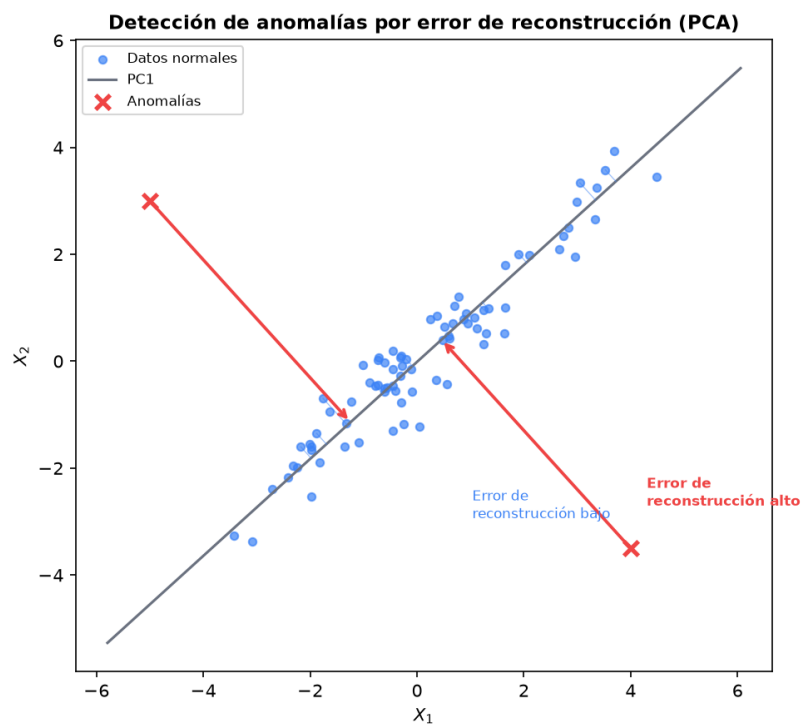


Figura10.2: Detección de anomalías mediante el error de reconstrucción de PCA.

10.4 Algoritmos específicos basados en no-supervisión

10.4.1 *Isolation forest* (bosque de aislamiento)

Es uno de los algoritmos [LTZ08] más eficientes y escalables para la detección de *outliers*, especialmente diseñado para trabajar en espacios de alta dimensionalidad.

- **Mecánica:** a diferencia de los métodos tradicionales que intentan modelar la densidad o los puntos normales, *isolation forest* busca aislar explícitamente cada observación. Para ello, construye un conjunto de árboles de decisión aleatorios. En cada nodo de un árbol, se selecciona una característica al azar y se elige un umbral de corte aleatorio (entre el mínimo y el máximo de esa variable) para dividir los datos en dos. Este proceso de partición recursiva continúa hasta que cada instancia queda aislada en su propia hoja.
- **Intuición:** dado que las anomalías se encuentran alejadas del grueso de la población de datos normales, requieren en promedio significativamente menos particiones aleatorias para ser aisladas. Por lo tanto, aquellas instancias que presenten una longitud de camino promedio más corta hacia la raíz a lo largo del bosque de árboles son catalogadas inmediatamente como anomalías.

Datos normales (densos) —————> Requieren muchas divisiones para aislarse.
 Anomalías (aisladas/raras) —————> Se aíslan rápidamente (pocas ramas).

La Fig. 10.3 compara ambos casos: la anomalía (izquierda) queda aislada con solo 2 divisiones aleatorias, mientras que un punto normal (derecha) necesita muchas más divisiones para separarse del resto.

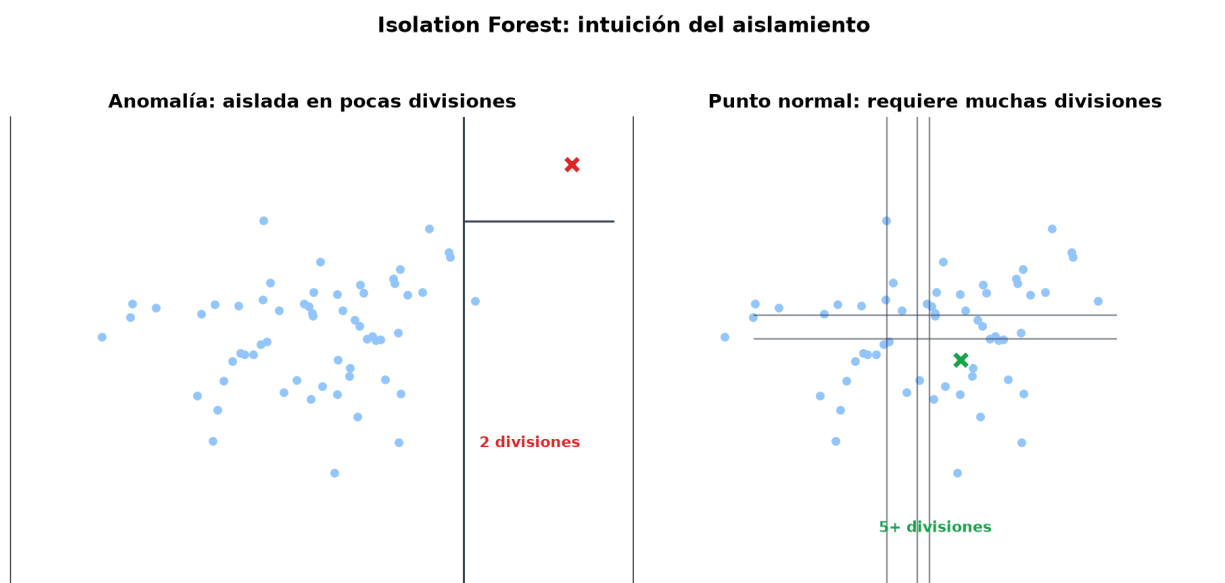


Figura 10.3: *Isolation Forest*: las anomalías se aíslan en menos particiones que los puntos normales.

10.4.2 Local Outlier Factor (LOF)

Este algoritmo [BKNS00] basa su funcionamiento en el análisis de la densidad local de las muestras utilizando un enfoque de vecinos más cercanos (K -NN).

- LOF compara la **densidad local** de una instancia con la densidad de sus vecinos más cercanos.
- Una instancia normal tendrá una densidad local similar a la de su entorno. En cambio, un **outlier local** presentará una densidad significativamente menor que la de sus vecinos más cercanos (estará más aislado en relación con la densidad de su vecindario inmediato).
- Este método es extremadamente útil para detectar **anomalías locales** que no destacarían en un análisis global porque sus valores absolutos no son extremos, pero sí resultan inusuales para el contexto específico de su clúster de pertenencia.

La Fig. 10.4 muestra un caso típico: el punto marcado en rojo no está lejos de todos los datos en términos absolutos, pero su densidad local es mucho menor que la de sus vecinos inmediatos, lo que lo delata como anomalía local.

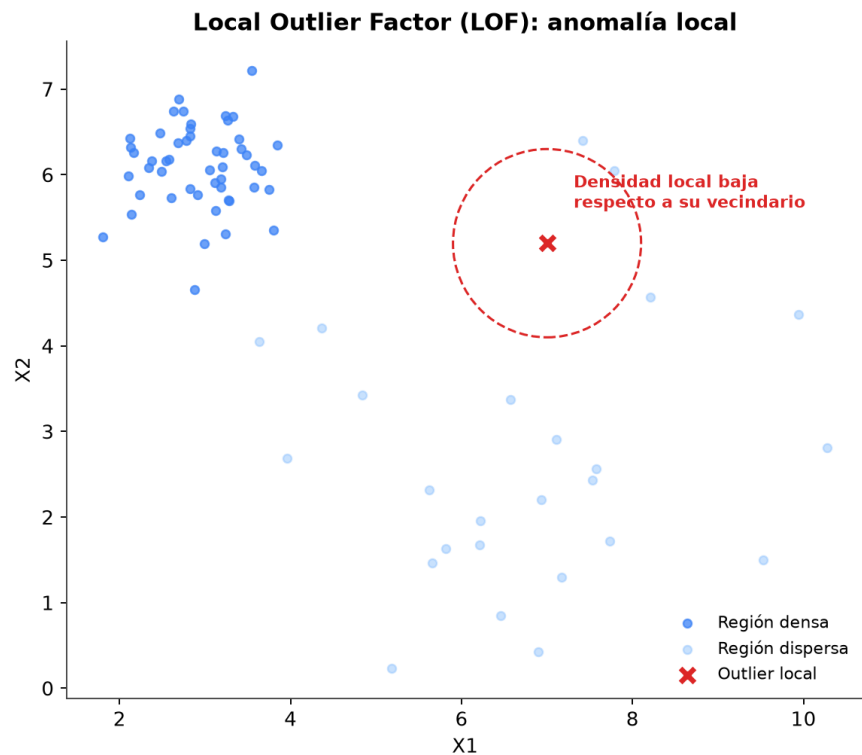


Figura 10.4: Local Outlier Factor: una anomalía local no destaca en un análisis global.

10.4.3 One-class SVM (máquinas de vectores de soporte de una clase)

Este algoritmo [ScholkopfPST+01] está optimizado específicamente para la detección de novedades (*novelty detection*) en escenarios donde se dispone de un conjunto de datos limpio para el entrenamiento.

- **Funcionamiento:** en lugar de buscar un hiperplano que separe dos clases, *one-class SVM* proyecta los datos a un espacio de características de alta dimensión mediante un *kernel* y busca separar las instancias de entrenamiento del origen.

- **Frontera de decisión:** esto equivale geoméricamente a encontrar la región o hipersfera de volumen mínimo que encierra a casi la totalidad de las muestras de entrenamiento. Si una nueva observación cae fuera de esta región delimitada por los vectores de soporte de frontera, es clasificada automáticamente como una anomalía o novedad.

La Fig. 10.5 muestra un ejemplo con datos de forma irregular: *one-class* SVM traza una frontera no lineal (morado) que envuelve ajustadamente la región de datos normales (azul), de modo que cualquier observación nueva que caiga fuera de ella (cruces rojas) se clasifica como anomalía.

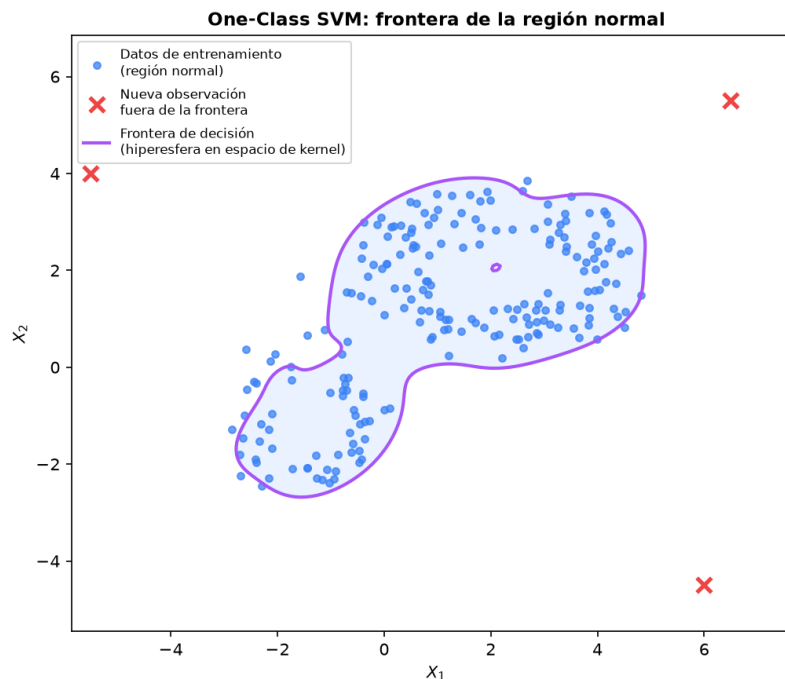


Figura10.5: *One-class* SVM: la frontera envuelve la región normal; lo que queda fuera se clasifica como anomalía.

10.5 Resumen

- **Anomalías:** eventos raros, contextuales o colectivos.
- **Métodos simples:** gaussiana para 1D, aplicación de PCA.
- **Métodos avanzados:** *Isolation Forest*, *Local Outlier Factor*, *one-class* SVM.

¡Felicidades! Has completado el contenido de “Fundamentos e introducción al aprendizaje automático”. Ahora tienes bases sólidas para explorar temas más avanzados como **aprendizaje profundo**, **procesamiento de lenguaje natural**, o **visión por computadora**.

Part IV

Bibliografía

BIBLIOGRAFÍA

Esta página recoge los libros de texto en los que se apoya este curso, así como los artículos y estudios originales citados a lo largo de los capítulos.

11.1 Libros de referencia del curso

11.2 Artículos y estudios citados

- [JWHT13] Gareth James, Daniela Witten, Trevor Hastie, and Robert Tibshirani. *An Introduction to Statistical Learning: with Applications in R*. Springer, New York, 2013. Freely distributed textbook for educational use (statlearning.com). doi:10.1007/978-1-4614-7138-7.
- [Bre01] Leo Breiman. Random forests. *Machine Learning*, 45(1):5–32, 2001. doi:10.1023/A:1010933404324.
- [BKNS00] Markus M. Breunig, Hans-Peter Kriegel, Raymond T. Ng, and Jörg Sander. Lof: identifying density-based local outliers. In *Proceedings of the 2000 ACM SIGMOD International Conference on Management of Data*, 93–104, 2000. doi:10.1145/342009.335388.
- [CV95] Corinna Cortes and Vladimir Vapnik. Support-vector networks. *Machine Learning*, 20(3):273–297, 1995. doi:10.1007/BF00994018.
- [EK SX96] Martin Ester, Hans-Peter Kriegel, Jörg Sander, and Xiaowei Xu. A density-based algorithm for discovering clusters in large spatial databases with noise. In *Proceedings of the Second International Conference on Knowledge Discovery and Data Mining (KDD-96)*, 226–231, 1996.
- [Fis36] Ronald A. Fisher. The use of multiple measurements in taxonomic problems. *Annals of Eugenics*, 7(2):179–188, 1936. doi:10.1111/j.1469-1809.1936.tb02137.x.
- [Fri01] Jerome H. Friedman. Greedy function approximation: a gradient boosting machine. *The Annals of Statistics*, 29(5):1189–1232, 2001. doi:10.1214/aos/1013203451.
- [Gau09] Carl Friedrich Gauss. *Theoria motus corporum coelestium in sectionibus conicis solem ambientium*. Friedrich Perthes und I.H. Besser, Hamburg, 1809.
- [HK70] Arthur E. Hoerl and Robert W. Kennard. Ridge regression: biased estimation for nonorthogonal problems. *Technometrics*, 12(1):55–67, 1970. doi:10.1080/00401706.1970.10488634.
- [Hot33] Harold Hotelling. Analysis of a complex of statistical variables into principal components. *Journal of Educational Psychology*, 24(6):417–441, 1933. doi:10.1037/h0071325.
- [KSH12] Alex Krizhevsky, Ilya Sutskever, and Geoffrey E. Hinton. Imagenet classification with deep convolutional neural networks. In *Advances in Neural Information Processing Systems*, volume 25, 1097–1105, 2012.

- [Leg05] Adrien-Marie Legendre. *Nouvelles méthodes pour la détermination des orbites des comètes*. Firmin Didot, Paris, 1805. Historical origin of the least-squares method.
- [LTZ08] Fei Tony Liu, Kai Ming Ting, and Zhi-Hua Zhou. Isolation forest. In *2008 Eighth IEEE International Conference on Data Mining*, 413–422, 2008. doi:10.1109/ICDM.2008.17.
- [Mac67] James MacQueen. Some methods for classification and analysis of multivariate observations. In *Proceedings of the Fifth Berkeley Symposium on Mathematical Statistics and Probability*, volume 1, 281–297, 1967.
- [MMRS06] John McCarthy, Marvin L. Minsky, Nathaniel Rochester, and Claude E. Shannon. A proposal for the dartmouth summer research project on artificial intelligence. *AI Magazine*, 27(4):12–14, 2006. Original proposal written in 1955; reprinted in *AI Magazine* in 2006. doi:10.1609/aimag.v27i4.1904.
- [MP43] Warren S. McCulloch and Walter Pitts. A logical calculus of the ideas immanent in nervous activity. *The Bulletin of Mathematical Biophysics*, 5(4):115–133, 1943. doi:10.1007/BF02478259.
- [MHM18] Leland McInnes, John Healy, and James Melville. Umap: uniform manifold approximation and projection for dimension reduction. 2018. URL: <https://arxiv.org/abs/1802.03426>, arXiv:1802.03426.
- [Pea01] Karl Pearson. On lines and planes of closest fit to systems of points in space. *Philosophical Magazine*, 2(11):559–572, 1901. doi:10.1080/14786440109462720.
- [Ros58] Frank Rosenblatt. The perceptron: a probabilistic model for information storage and organization in the brain. *Psychological Review*, 65(6):386–408, 1958. doi:10.1037/h0042519.
- [Rou87] Peter J. Rousseeuw. Silhouettes: a graphical aid to the interpretation and validation of cluster analysis. *Journal of Computational and Applied Mathematics*, 20:53–65, 1987. doi:10.1016/0377-0427(87)90125-7.
- [RHW86] David E. Rumelhart, Geoffrey E. Hinton, and Ronald J. Williams. Learning representations by back-propagating errors. *Nature*, 323(6088):533–536, 1986. doi:10.1038/323533a0.
- [ScholkopfPST+01] Bernhard Schölkopf, John C. Platt, John Shawe-Taylor, Alex J. Smola, and Robert C. Williamson. Estimating the support of a high-dimensional distribution. *Neural Computation*, 13(7):1443–1471, 2001. doi:10.1162/089976601750264965.
- [Tib96] Robert Tibshirani. Regression shrinkage and selection via the lasso. *Journal of the Royal Statistical Society: Series B*, 58(1):267–288, 1996. doi:10.1111/j.2517-6161.1996.tb02080.x.
- [TWH01] Robert Tibshirani, Guenther Walther, and Trevor Hastie. Estimating the number of clusters in a data set via the gap statistic. *Journal of the Royal Statistical Society: Series B*, 63(2):411–423, 2001. doi:10.1111/1467-9868.00293.
- [Tur50] Alan M. Turing. Computing machinery and intelligence. *Mind*, 59(236):433–460, 1950. doi:10.1093/mind/LIX.236.433.
- [vdMH08] Laurens van der Maaten and Geoffrey Hinton. Visualizing data using t-sne. *Journal of Machine Learning Research*, 9:2579–2605, 2008.
- [VSP+17] Ashish Vaswani, Noam Shazeer, Niki Parmar, Jakob Uszkoreit, Llion Jones, Aidan N. Gomez, Łukasz Kaiser, and Illia Polosukhin. Attention is all you need. In *Advances in Neural Information Processing Systems*, volume 30, 2017.